
Software Design Documentation

for

FixUp Pro System

Version 1.0 approved

Prepared by

**CB22136 AZLEEN FARISYA BINTI AHMAD FAUZI
CB22045 UMIE KALSUM BINTI GHAZALI
CB23134 MUHAMMAD NASYRIQ NAIM BIN MOHD NADZIR
CB23145 MOHAMAD HILMAN NAFIS BIN MOHD AFFENDEY**

AgileTech.Co

- 2025

Table of Content

1.0 INTRODUCTION	3
1.1 Purpose of Document	3
1.2 System Identification	3
1.3 System Overview	4
1.4 Module Overview	5
2.0 REFERENCE DOCUMENT	7
3.0 SYSTEM ARCHITECTURE	8
3.1 General Architecture	8
3.1.1 Application Layer	9
3.1.1.1 Module Manage Registration [FPS-PACK-100]	9
3.1.1.2 Module Manage User Profile [FPS-PACK-200]	9
3.1.1.2 Module Manage Working Schedule [FPS-PACK-300]	10
3.1.1.4 Module Manage Inventory [FPS-PACK-400]	11
3.1.1.5 Module Manage Payment [FPS-PACK-500]	12
3.1.1.6 Module Manage Rating [FPS-PACK-600]	13
3.1.2 Business Layer	14
3.1.2.1 Provider [FPS-PACK-700]	14
3.1.2.2 Domain [FPS-PACK-800]	15
3.1.3 Middleware Layer	17
3.1.3.1 API [FPS-PACK-900]	17
3.2 Package Module	18
3.2.1 Module Manage Registration	18
3.2.2 Module Manage User Profile	19
3.2.3 Module Manage Working Schedule	21
3.2.4 Module Manage Inventory	22
3.2.5 Module Manage Payment	24
3.2.6 Module Manage Rating	26
3.3 Data Dictionary	29
4.0 DETAIL DESIGN	33
4.1 Manage Registration and User Profile	33
4.2 Manage Schedule	43
4.3 Manage Inventory	57
4.4 Manage Payment	70
4.5 Manage Rating	75
5.0 REQUIREMENT TRACEABILITY	80

1.0 INTRODUCTION

1.1 Purpose of Document

This Software Design Document (SDD) outlines a detailed process for developing the FixUp Pro System. It describes the design of the system, including important elements like work ratings, inventory control, foreman scheduling, and payment processing. The document describes how these elements will cooperate to provide a seamless and effective user experience.

Important factors related to performance, security, and maintainability are also included in the SDD. It ensures consistency and alignment throughout the development process by acting as a reference for all team members, including developers, designers, testers, and project managers. This document facilitates the effective delivery of a dependable, user-focused system by providing clear definitions for the design and scope, which helps to minimize misunderstandings.

1.2 System Identification

System ID	FPS-2025-V1.0
Description	<System Abbreviation> - <Year Development> - <Version>

Document ID	FPS-SDD-2025-V1.0
Description	<System Abbreviation> - <Software Design Documentation> - <Year Development> - <Version>

Package ID	FPS-PACK-100
Description	<System Abbreviation> - <Package> - <Package Number>

Class ID	FPS-PACK-101
Description	<System Abbreviation> - <Package> - <Class Number>

1.3 System Overview

The FixUp Pro System is a feature-rich platform designed specifically for workshop management in the gig economy. By providing safe registration and user profile management, it allows gig foremen and workshop owners to maintain up-to-date information while maintaining role-based access control. The Manage Working Schedule section of the system makes job planning easier by enabling users to create, edit, and monitor assignments effectively. Automated notifications and real-time updates are two features that improve process collaboration and help prevent schedule conflicts.

FixUp Pro also facilitates essential workshop operations including payments, inventory management, and performance tracking. With features for inventory reporting and restocking notifications, the Manage Inventory module aids in precisely monitoring stock levels. The Manage Payment module streamlines financial transactions by enabling equipment purchases directly within the system. Lastly, the Manage Rating module promotes a professional work environment by enabling workshop owners to assess gig foremen on the basis of dependability and service quality. When combined, these modules increase output, responsibility, and transparency, which makes FixUp Pro a reliable option for contemporary workshop operations.

1.4 Module Overview

The **Registration Module** enables users to create accounts based on their roles such as workshop owners and foremen. During the registration process, users are required to enter personal information including name, email, password and role selection. The system performs input validation to ensure data integrity and completeness before accepting the registration. This module ensures that user accounts are accurately categorized to allow tailored access to features based on user roles. Only valid and properly formatted registrations are allowed into the system to ensure security.

The **User Profile Module** allows registered users to view and manage their personal information. Each profile contains details such as name, contact information, role, IC number, address, gender and profile image. Users can update their profile data as needed through an intuitive interface to ensure that their information remains current. This module supports both viewing and editing of user data, contributing to user and account personalization while keeping the system's user records up to date and relevant.

The **Manage Working Schedule** module simplifies workshop operations by allowing owners to allocate jobs to foremen with specific time slots, while foremen may establish their availability and update job statuses (pending, in-progress, and completed). In order to avoid double booking, it has a centralized schedule list, an interactive calendar display, and conflict detection. Foremen can record their work hours and receive notifications, while owners can monitor progress in real time. The module optimizes workshop efficiency in the gig economy by ensuring effective coordination, automated reminders, and transparent communication between both parties. It is built with Firebase for data storage and Flutter for cross-platform accessibility.

The **Manage Inventory** module enables workshop owners to effectively track, update, and organize their stock of parts, tools, and supplies. This module provides essential inventory functions such as adding new items, editing existing entries, and deleting obsolete or damaged stock. Each item can be tagged with details like quantity, category, price, brand and model name, making it easy to categorize and locate. A key highlight of this module is the built-in ability to send or request inventory from other workshops through an integrated marketplace feature. This promotes resource sharing, reduces wastage, and improves operational readiness, especially in times of urgent demand. Additionally, the system logs every import request parts, allowing workshop owners to review their import request from other workshop. The module is designed to be mobile-responsive and user-friendly, ensuring that inventory management tasks can be completed easily on-site or remotely. By streamlining how workshop supplies are controlled and monitored, this module plays a crucial role in reducing downtime and ensuring smooth daily operations.

The **Manage Payment** module for the FixUp Pro System streamlines the process of managing financial transactions between workshop owners and foremen. This module allows workshop owners to initiate payments for completed tasks, ensuring a smooth and timely settlement process. It integrates with external payment gateways to facilitate secure online payment processing, supporting multiple methods such as bank transfers and e-wallets. The system provides transparency by maintaining detailed payment histories and transaction records for both parties. Additionally, it includes features for payment validation, refund processing, and notifications for pending and overdue payments. By leveraging secure payment gateway integration, the module guarantees compliance with data protection regulations, delivering a secure and efficient payment experience for all users.

The **Rating Module** facilitates feedback between users, specifically enabling workshop owners to rate and comment on their foremen's performance. Ratings include a numeric score and optional comments to help build a transparent performance tracking system. The module ensures ratings are linked to both the rater and the rated user, maintaining accountability. Users can submit new ratings, edit existing ones and view a dashboard of all ratings they've given or received. This promotes constructive feedback, helps monitor service quality and fosters trust within the platform.

2.0 REFERENCE DOCUMENT

Techdynasty. (2025, May 1). Design Patterns in Flutter - TechDynasty - Medium. Medium. <https://techdynasty.medium.com/design-patterns-in-flutter-25191278149c>

Reddix, A. (2024, May 9). Top 10 Design Patterns in Flutter: A Comprehensive Guide. DEV Community. <https://dev.to/aaronreddix/top-10-design-patterns-in-flutter-a-comprehensive-guide-50ca>

Huat, K. L. C. (2021, December 13). Flutter Provider Architecture with Firebase Full Guide — by building a working CRUD app. Medium. <https://medium.com/fluttercommunity/flutter-provider-architecture-with-firebase-full-guide-by-building-a-working-crud-app-4824cec04e59>

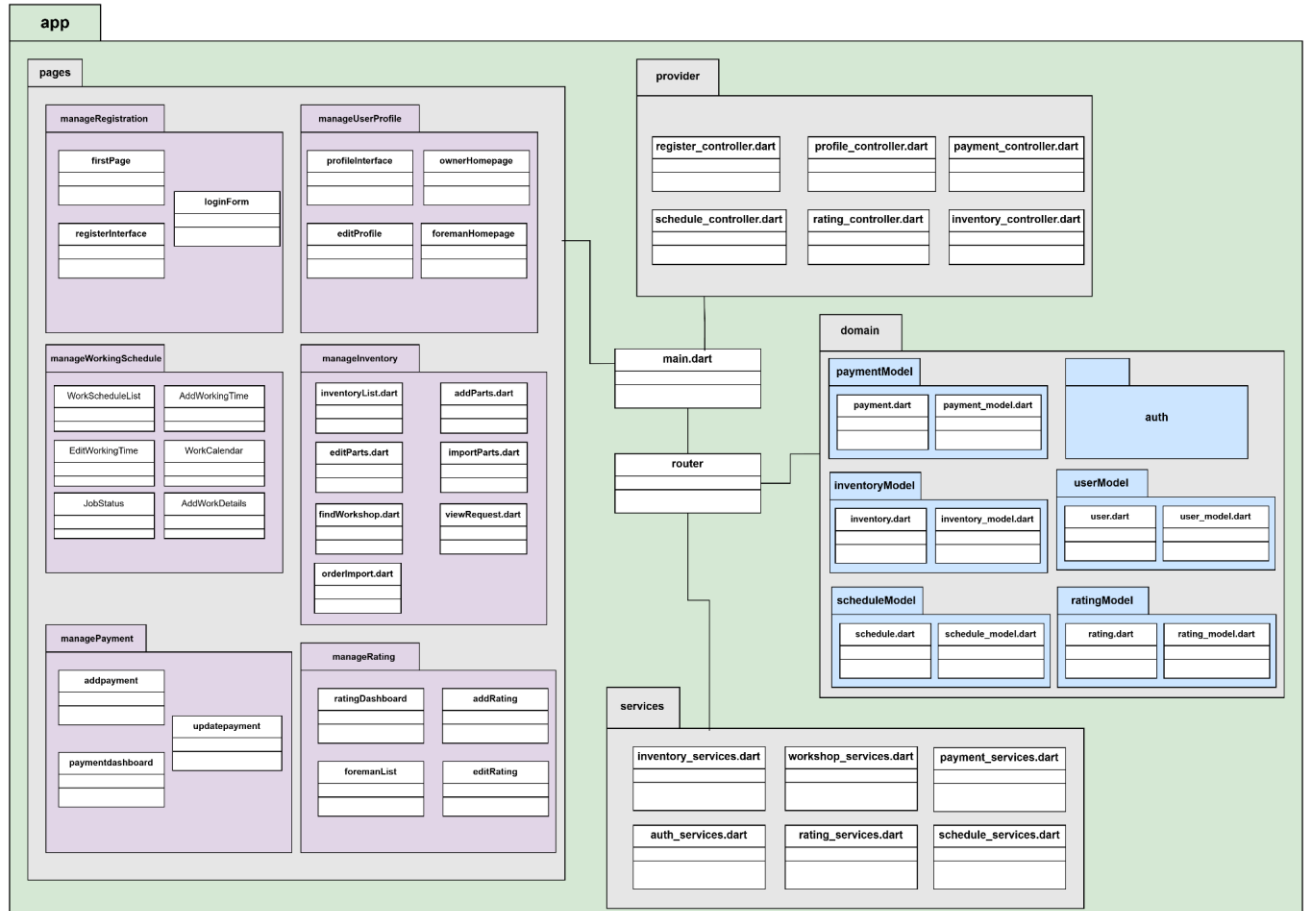
Bizzotto, A. (2021, August 31). Starter Architecture for Flutter & Firebase Apps using Riverpod. <https://codewithandrea.com/videos/starter-architecture-flutter-firebase/>

IEEE. (2009). Software Design Document (SDD) Template (summarized from IEEE STD 1016). In IEEE Recommended Practice for Software Design Descriptions. https://wildart.github.io/MISG5020/standards/SDD_Template.pdf freeCodeCamp . (2018, July 13) .

How to write a good software design doc. [freeCodeCamp.org](https://www.freecodecamp.org/news/how-to-write-a-good-software-design-document-66fcf019569c/). <https://www.freecodecamp.org/news/how-to-write-a-good-software-design-document-66fcf019569c/>

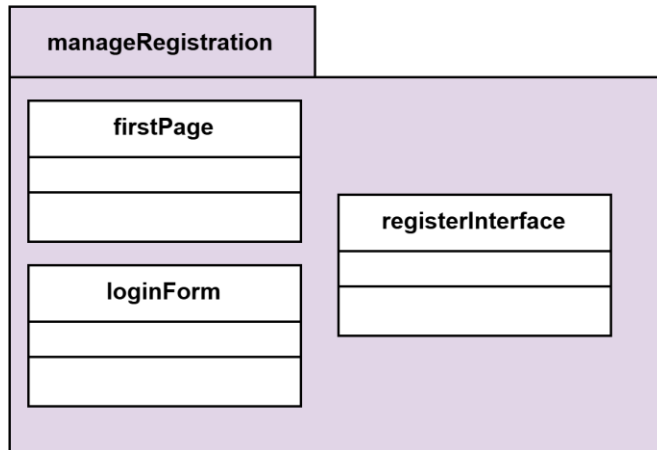
3.0 SYSTEM ARCHITECTURE

3.1 General Architecture



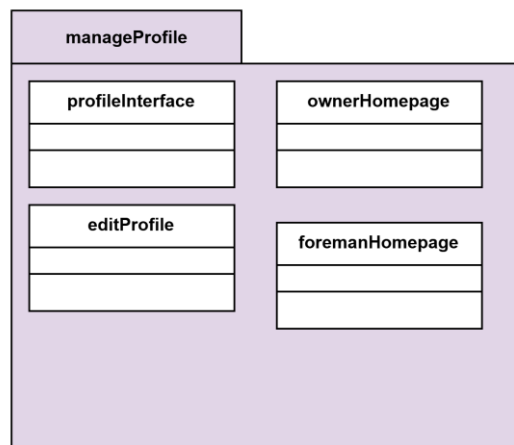
3.1.1 Application Layer

3.1.1.1 Module Manage Registration [FPS-PACK-100]



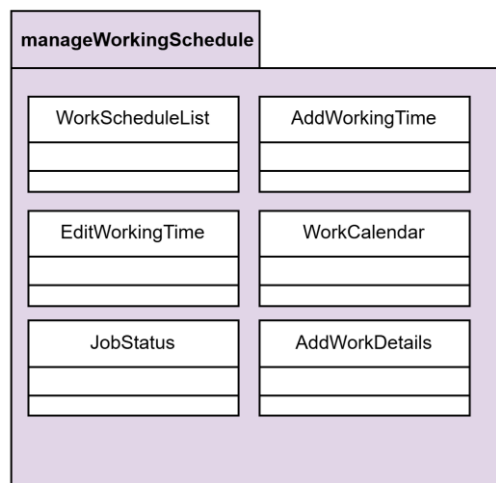
Class Name	Description
firstPage [FPS-PACK-101]	This class shows the first screen of the app where users can choose to log in or register
loginForm [FPS-PACK-102]	This class shows the login form where users enter their username and password to log into the app
registerInterface [FPS-PACK-103]	This class shows the registration form where new users enter their details to create an account

3.1.1.2 Module Manage Profile [FPS-PACK-200]



Class Name	Description
profileInterface [FPS-PACK-201]	This class shows the user profile with details like name, email, phone, and role. It also lets users go to the edit page
editProfile [FPS-PACK-202]	This class shows a form where users can edit and save their profile information
ownerHomepage [FPS-PACK-203]	This class shows a homepage for owner
foremanHomepage [FPS-PACK-204]	This class shows a homepage for foreman

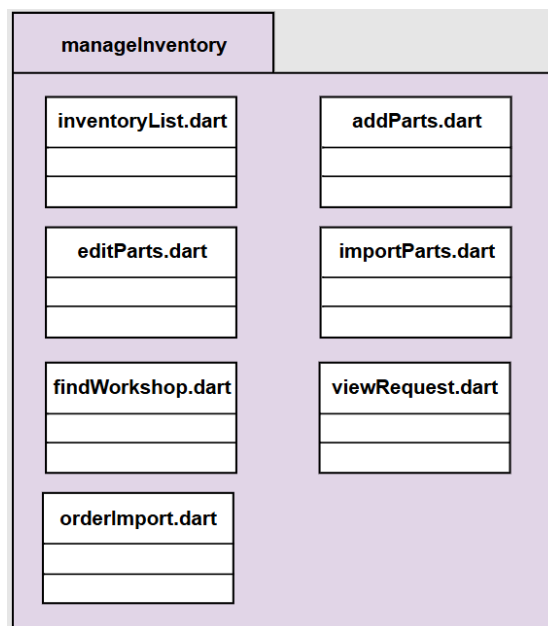
3.1.1.2 Module Manage Working Schedule [FPS-PACK-300]



Class Name	Description
WorkScheduleList [FPS-PACK-301]	This class shows the foreman work schedule list when clicking on the calendar and details that have been assigned.
AddWorkingTime [FPS-PACK-302]	This class shows the add working time form where users input their availability time into the system.
EditWorkingTime [FPS-PACK-303]	This class shows the edit and delete working time form where users input their availability time into the system.
WorkCalendar [FPS-PACK-304]	This class shows the work list of a foreman that has been assigned.

JobStatus [FPS-PACK-305]	This class shows the job status of a foreman where users input their job status whether complete or pending, to be seen by the workshop owner.
AddWorkingDetails [FPS-PACK-306]	This class shows the working details form where the user needs to input working details that the foreman needed to do.

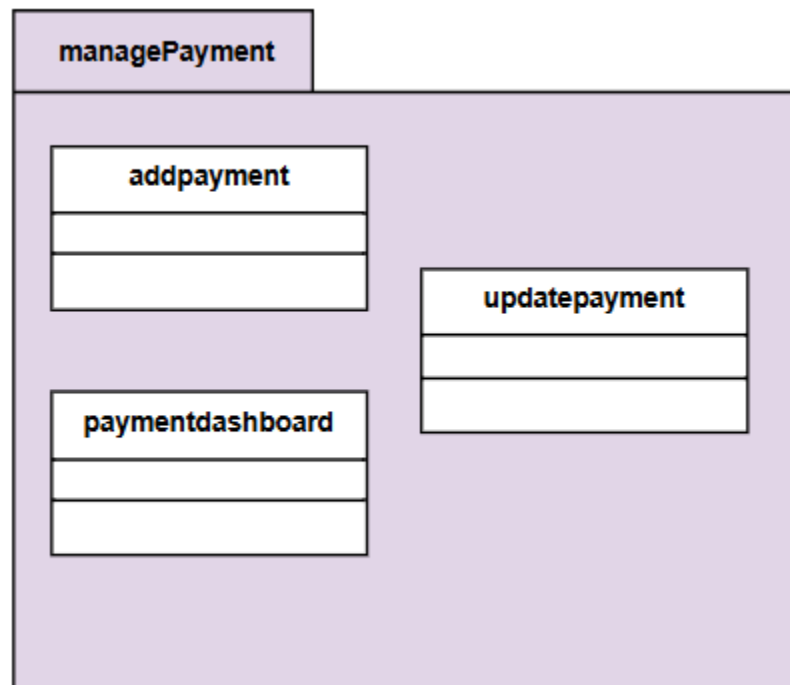
3.1.1.4 Module Manage Inventory [FPS-PACK-400]



Class Name	Description
inventoryList [FPS- PACK-401]	Represents the main list view of all inventory items owned by the workshop owner and also the imported parts from other workshop. It provides access to item details, image, current stock quantity, price and available actions such as add parts, edit parts, import parts and category filter.
addparts [FPS- PACK-402]	Handles the functionality for adding new items to the inventory. This class manages input validation, image, item categorization, brand and model name, quantity, and price.
editParts [FPS- PACK-403]	Allows users to modify existing inventory item details, such as updating quantity, price, editing brand and model name, or image and category.

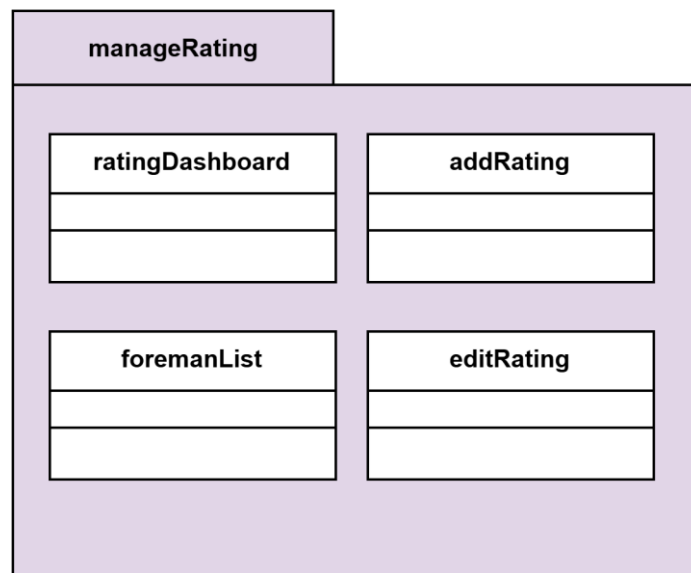
importParts [FPS- PACK-404]	Manages the list of incoming or outgoing part requests between workshops. It shows the status of each request like pending request or approved.
findWorkshop [FPS- PACK-405]	Provides functionality to search for other workshops within the marketplace. This class supports filtering by location and availability of requested parts.
orderImport [FPS- PACK-406]	Displays the order request where the workshop owner can view part details from other workshops. This class shows information about the selected part, including workshop details, item name, available quantity, brand, and price. It allows to select the quantity and make an order request to the workshop.
viewRequest [FPS- PACK-407]	Displays detailed information of a selected list import part request, showing item details, request status, and response options such as cancel request and approve.

3.1.1.5 Module Manage Payment [FPS-PACK-500]



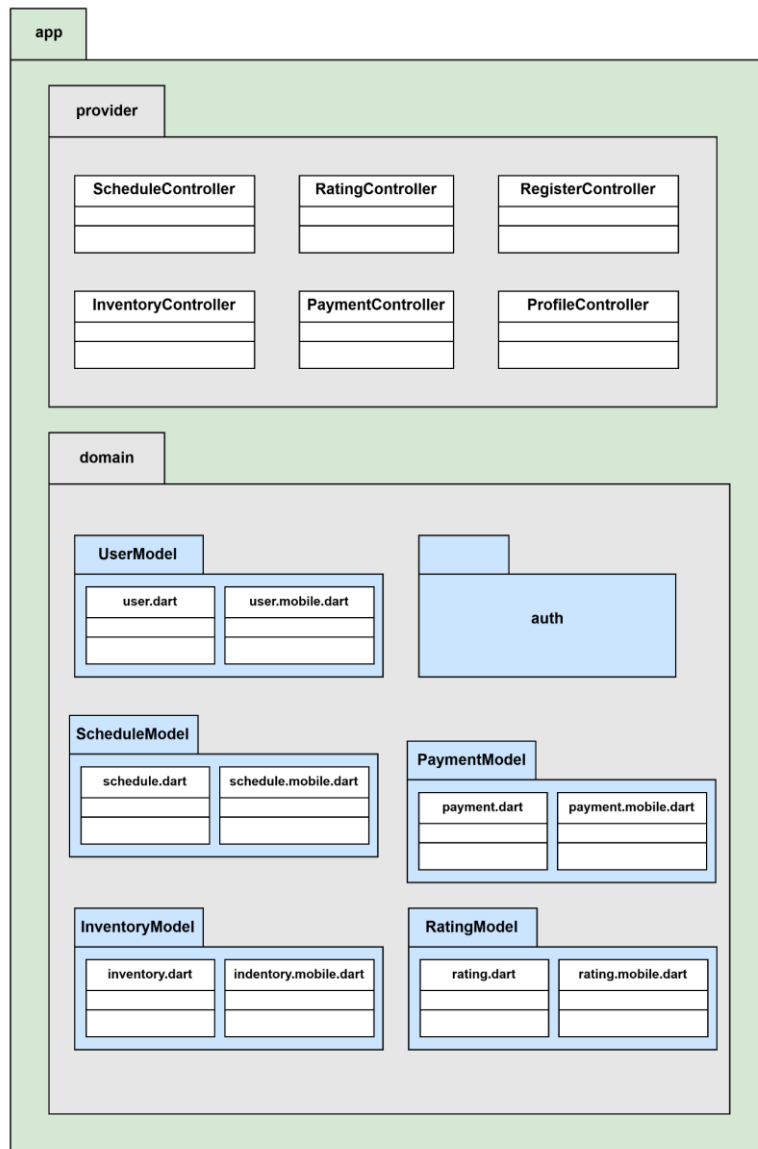
Class Name	Description
Paymentdashboard [FPS-PACK-501]	Displays the payment dashboard for workshop owners and foremen, showing pending and completed payments with task and user details.
addpayment [FPS-PACK-502]	Provides the interface for workshop owners to enter and submit payment details, including payment method and amount.
Updatepayment [FPS-PACK-503]	Provides interface for workshop owner to update payment.

3.1.1.6 Module Manage Rating [FPS-PACK-600]



Class Name	Description
ratingDashboard [FPS-PACK-601]	This class shows a summary of all ratings, including foreman names and scores
foremanList [FPS-PACK-602]	This class displays a list of foremen that users can select to rate or edit.
addRating [FPS-PACK-603]	This class shows a form where users can give a new rating and comment for a selected foreman.
editRating [FPS-PACK-604]	This class allows users to update an existing rating and comment they previously submitted.

3.1.2 Business Layer



3.1.2.1 Provider [FPS-PACK-700]

Class Name	Description
register_controller [FPS-PACK-701]	The controller that manages all registration actions including handling user input, validating data and sending registration details to the database
profile_controller [FPS-PACK-702]	The controller that handles profile actions like viewing, updating and saving user profile

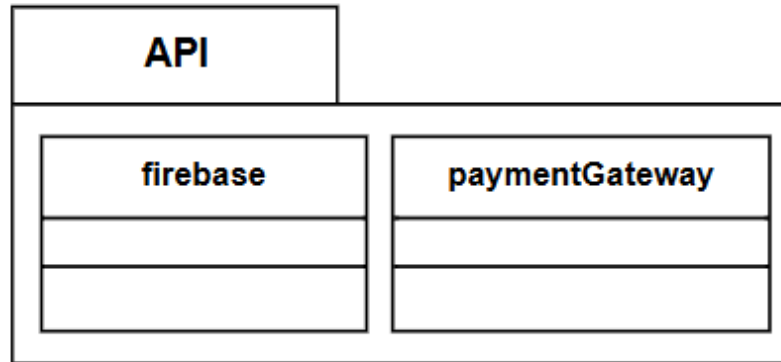
	information
schedule_controller [FPS-PACK-703]	The controller that handles all working schedule management involving the working availability time, work schedule list, work calendar and job assignment status.
inventory_controller [FPS-PACK-704]	The controller is responsible for handling all inventory-related actions such as listing items, adding new items, editing stock details, and processing marketplace requests between workshops. It validates inputs and interacts with inventory models for data persistence.
payment_controller [FPS-PACK-705]	The PaymentController class handles all payment operations in the system. It manages the process of initiating, validating, and recording payments.
rating_controller [FPS-PACK-706]	This controller manages all rating actions such as adding, editing and viewing ratings. It handles input validation and connects the user interface with the data layer

3.1.2.2 Domain [FPS-PACK-800]

Class Name	Description
userModel [FPS-PACK-801]	This entity class represents the user data model. It stores user-related information such as name, email, phone number, password, and role (Workshop Owner or Foreman).
scheduleModel [FPS-PACK-802]	The model stores data involving all the working schedule management data including time, days, work list and availability of the user.
inventoryModel [FPS-PACK-803]	Stores data related to the workshop owner inventory and also inventory import part requests between workshops. It includes sender and receiver workshop IDs, requested item details, quantity, and request status.
paymentModel [FPS-PACK-804]	This class manages how payments work. It

	handles payment requests, checking payment details are correct, connecting payment systems, and keeping payment up to date.
ratingModel [FPS-PACK-805]	This class defines the data model for a rating. It stores information such as rating ID, user ID, rating score, comment, and date. It is used to represent and process rating data in the system.

3.1.3 Middleware Layer

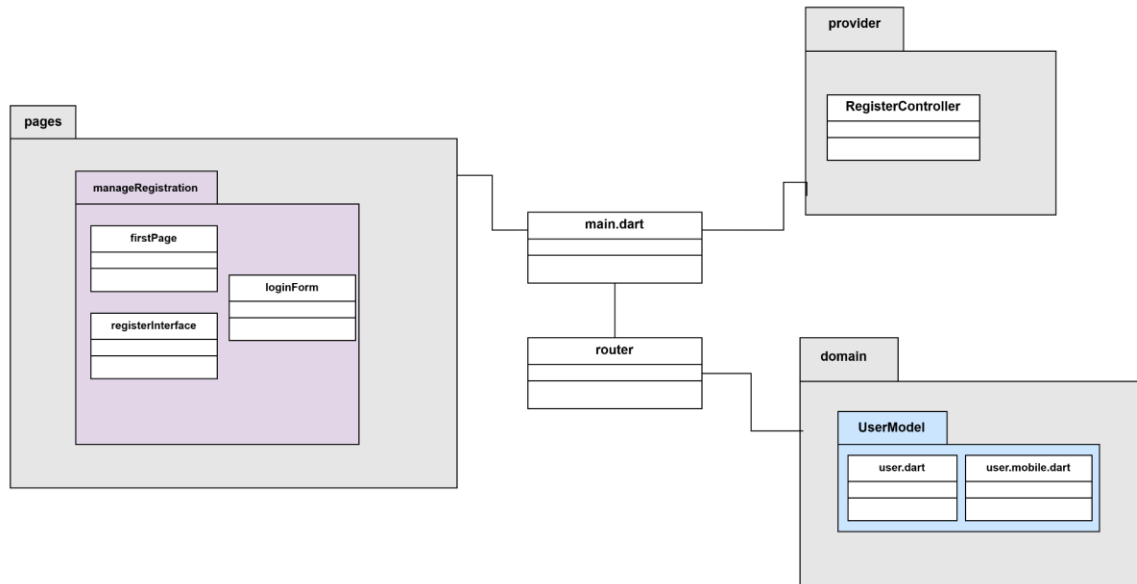


3.1.3.1 API [FPS-PACK-900]

Class Name	Description
Firestore [FPS-PACK-901]	A cloud-based platform that helps developers build and run web or mobile apps, offering a variety of tools and services like cloud-based hosting.
paymentGateway [FPS-PACK-902]	The intermediary between user's bank and the merchant's payment processor, facilitating a secure online transaction.

3.2 Package Module

3.2.1 Module Manage Registration



Type of Class	Boundary	Class Name	firstPage
Description	This class displays the first screen of the app. It provides users with options to either log in or register. It acts as the starting point for user interaction and navigation based on their choice.		

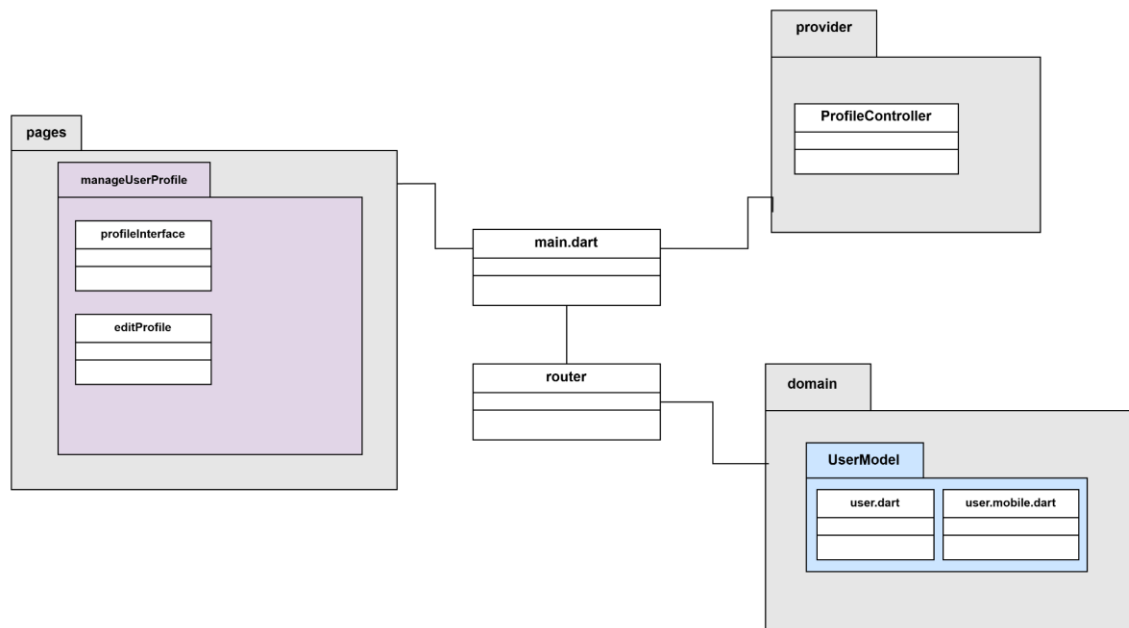
Type of Class	Boundary	Class Name	loginForm
Description	This class displays the login form where users enter their email and password. It includes validation for user input and navigates users to their respective dashboards upon successful login.		

Type of Class	Boundary	Class Name	registerInterface
Description	This class provides the user interface for account registration. It includes form fields to collect user details such as name, email, password, phone number, and role (Workshop Owner or Foreman), along with input validation and a sign-up button.		

Type of Class	Controller	Class Name	RegisterController
Description	This controller handles all actions related to user registration. It receives input from the registration interface, validates the data, interacts with the user entity or database to create new accounts, and returns appropriate success or error responses.		

Type of Class	Entity	Class Name	User
Description	This entity class represents the user data model. It stores user-related information such as name, email, phone number, password, and role (Workshop Owner or Foreman). It is used for data processing, storage, and interaction with the registration and login features.		

3.2.2 Module Manage Profile



Type of Class	Boundary	Class Name	profileInterface
Description	This class displays the user's profile information, including name, email, phone number, and role. It provides a user-friendly interface for viewing personal details and includes an option to navigate to the edit		

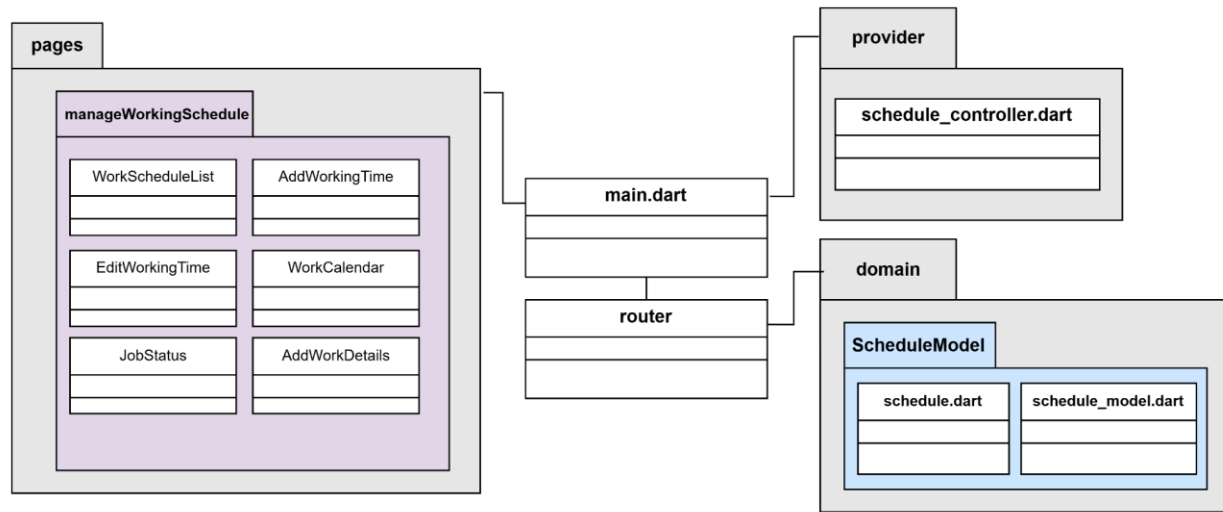
	profile screen.
--	-----------------

Type of Class	Boundary	Class Name	editProfile
Description	This class provides the interface that allows users to update their profile information. It includes editable form fields, input validation, and a save button that sends the updated data to the controller for processing.		

Type of Class	Controller	Class Name	ProfileController
Description	This controller manages profile-related operations such as retrieving user data, handling profile updates, validating changes, and saving updated information to the database. It serves as the link between the user interface and the data layer.		

Type of Class	Entity	Class Name	User
Description	This entity class represents the user data model. It stores user-related information such as name, email, phone number, password, and role (Workshop Owner or Foreman). It is used for data processing, storage, and interaction with the registration and login features.		

3.2.3 Module Manage Working Schedule



Type of Class	Boundary	Class Name	WorkScheduleList
Description	This class displays the work schedule list of the user. Users can view the foreman that availables on the day by clicking on a date in the calendar.		

Type of Class	Boundary	Class Name	AddWorkingTime
Description	This class displays the add working time form where users can input their time availability into the system to be adjusted in their work schedule.		

Type of Class	Boundary	Class Name	EditWorkingTime
Description	This class displays the edit working time form where users can update and delete their time availability into the system to be adjusted in their work schedule.		

Type of Class	Boundary	Class Name	WorkCalendar
Description	This class displays the all work list that needs to be done.		

Type of Class	Boundary	Class Name	JobStatus
Description	This class displays the job status for the user to input to track whether		

	all the work is completed or not completed in the system.
--	---

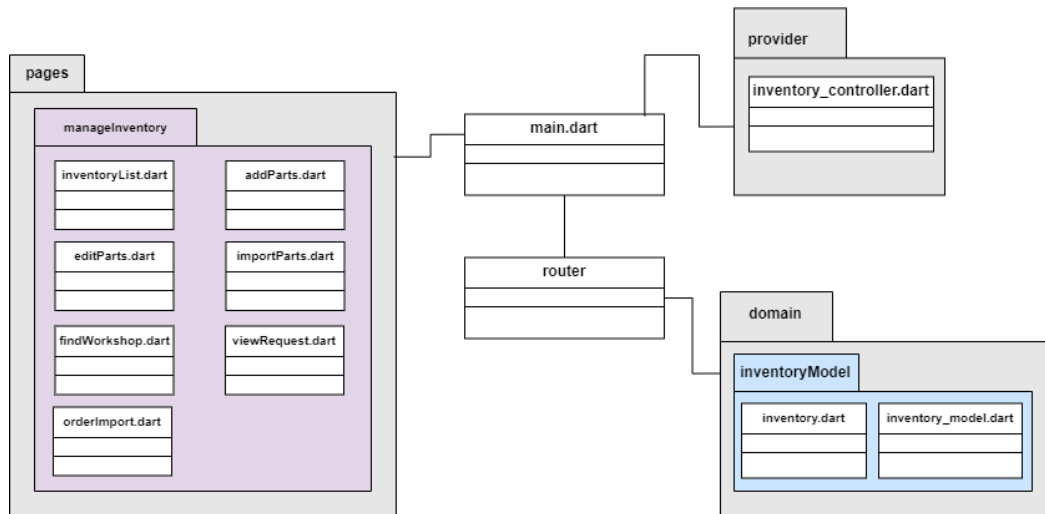
Type of Class	Boundary	Class Name	AddWorkDetails
Description	This class displays the add work details form where users can input the work details like vehicle details and job description. .		

Type of Class	Controller	Class Name	shedule_controller.dart
Description	This class handles all working schedule management involving the working availability time, work schedule list, work calendar and job assignment status.		

Type of Class	Entity	Class Name	schedule_model.dart
Description	The model represents data for the schedule data model. It stores work schedule information such as date, time, vehicle details and job assignment. It is used for data processing, storage, and interaction with the registration and login features.		

Type of Class	Entity	Class Name	schedule.dart
Description	The model represents data integration involving all the working schedule management data including time, days, work list and availability of the user.		

3.2.4 Module Manage Inventory



Type of Class	Boundary	Class Name	inventoryList
Description	This class displays the full list of inventory items currently available in the workshop. Users can view item names, quantities, images, brands, prices, and categories. It includes search and filter functionality and acts as the main interface for accessing inventory details.		

Type of Class	Boundary	Class Name	addParts
Description	This class provides the user interface for adding new inventory items. It includes form fields for entering item name, brand, model, category, quantity, and price, along with validation to ensure input is numeric and complete. Image selection is enabled based on category, and it provides success or error messages after submission.		

Type of Class	Boundary	Class Name	editParts
Description	This class displays the interface to edit existing inventory items. It pre-fills item data for editing and allows users to update item name, quantity, brand, category, price, and image. The form includes numeric validation and displays feedback messages for user actions.		

Type of Class	Boundary	Class Name	importParts
---------------	----------	------------	-------------

Description	This class shows a list of incoming and outgoing part requests between workshops. It includes the request status (pending, approved,cancelled), the date and time of requests, and allows navigation to view request details.
-------------	---

Type of Class	Boundary	Class Name	findWorkshop
Description	This class provides a search interface for locating workshops within the marketplace. It allows filtering by location or part availability, shows available parts with prices, and includes workshop information with an option to place a direct order.		

Type of Class	Boundary	Class Name	orderImport
Description	This class shows the order request interface where the workshop owner can view part details from other workshops, including workshop details, item name, quantity, brand, and price. It allows selecting order quantity, validates against available stock, calculates total price.		

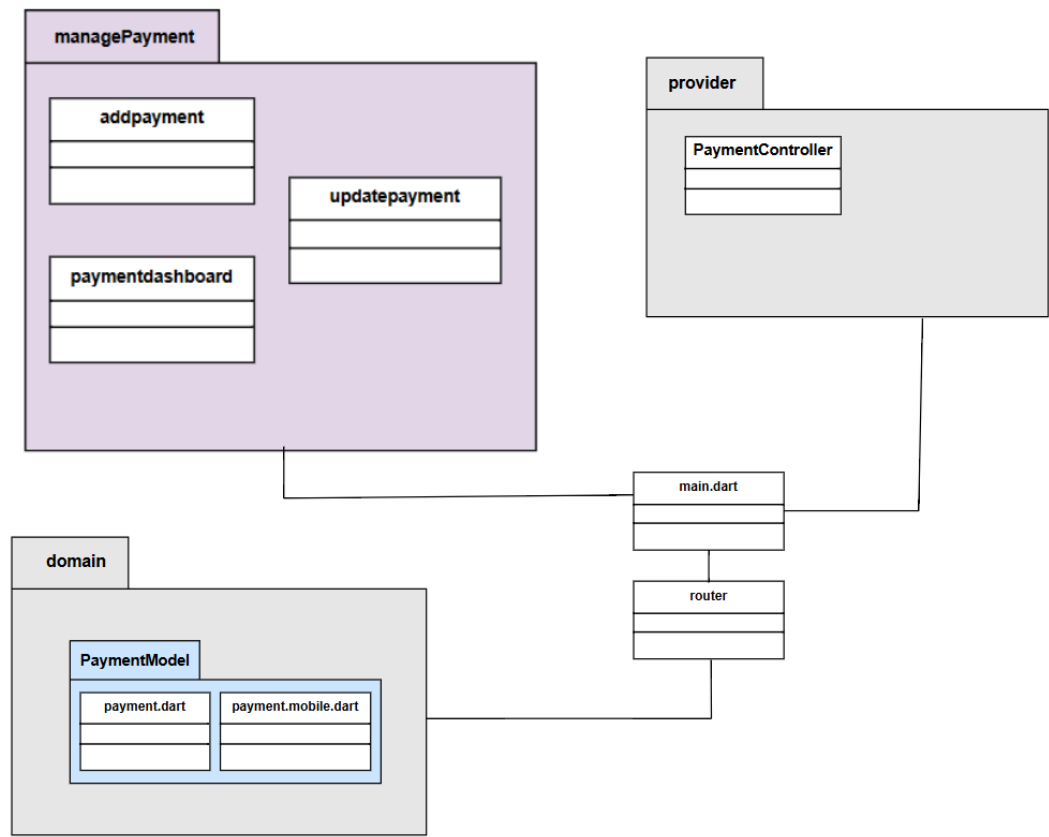
Type of Class	Boundary	Class Name	viewRequest
Description	This class displays detailed information about a selected part request, including item details, workshop information, requested quantity, and status. It shows the request date and time, calculates total price, allows cancellation or approval.		

Type of Class	Controller	Class Name	inventory_controller
Description	This controller handles all actions related to inventory management. It receives inputs from boundary classes (add, edit, delete, request), validates and processes them, and interacts with Firebase for database operations. It also manages user authentication, handles exceptions, and provides search functionality.		

Type of Class	Entity	Class Name	InventoryModel
---------------	--------	------------	----------------

Description	This entity class represents the structure of an inventory item. It includes ID, brand, model, category, price, quantity, image URL, and description. It provides serialization methods (toMap, fromMap) and is used for data transfer between the UI and database.
-------------	---

3.2.5 Module Manage Payment



Type of Class	Boundary	Class Name	paymentdashboard
Description	This class represents the view of the payment dashboard for both workshop owners and foremen. It displays details of paid and unpaid tasks with payment information, such as user details, cleaning tasks, payment IDs, amounts, statuses, and dates.		

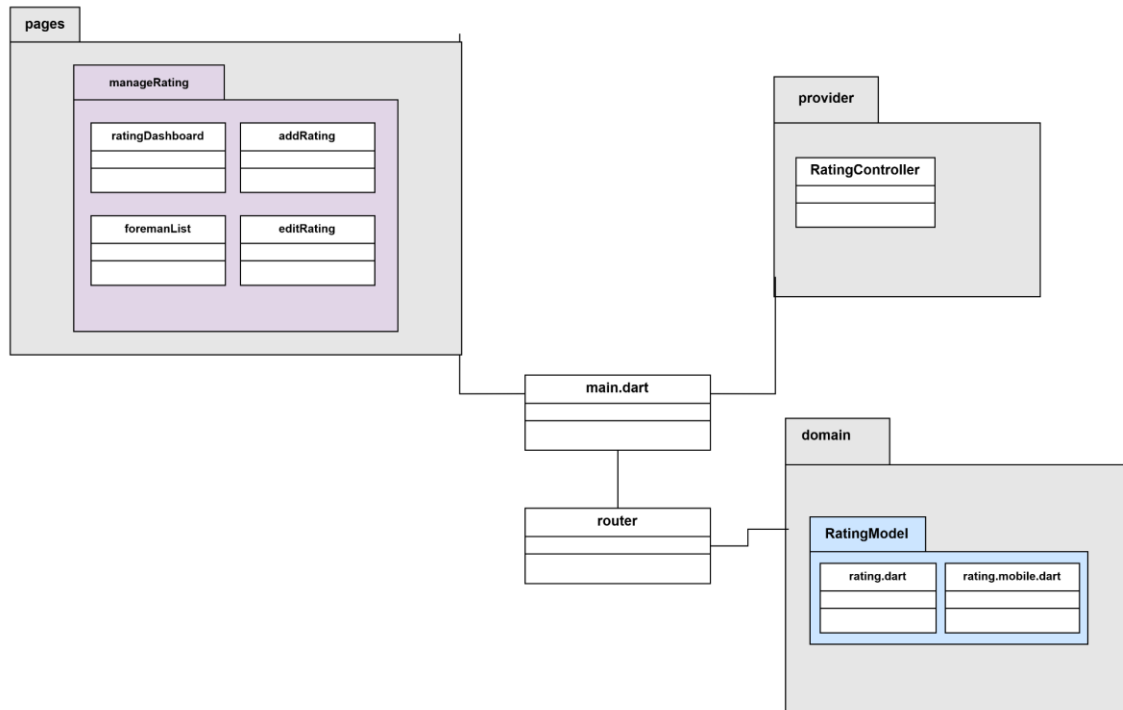
Type of Class	Boundary	Class Name	addpayment
Description	This class handles the view for payment entry, where the workshop owner can input payment details such as payment method, amount and account information.		

Type of Class	Boundary	Class Name	updatepayment
Description	This class provides a confirmation message and detailed payment information, including the payment amount, payment ID, task information, and the payment status (successful). It gives the user a chance to view the details of the completed transaction and navigate back to the payment dashboard or transaction history.		

Type of Class	Controller	Class Name	paymentController
Description	This class is responsible for managing the flow of payment transactions. It acts as an intermediary between the user interface (views) and the payment gateway in the backend.		

Type of Class	Entity	Class Name	Payment
Description	This class represents the payment entity and stores payment details such as payment ID, amount, date, task, foreman details, payment method, and status.		

3.2.6 Module Manage Rating



Type of Class	Boundary	Class Name	ratingDashboard
Description	This class displays a dashboard view of all ratings submitted by the workshop owner. It shows an overview of rated foremen		

Type of Class	Boundary	Class Name	foremanList
Description	This class displays a list of foremen that users can rate and edit. It includes basic foreman details such as name and profile to help users choose whom to rate.		

Type of Class	Boundary	Class Name	addRating
Description	This class provides the interface for users to submit a new rating for a foreman. It includes input fields for rating score, comments and submission buttons		

Type of Class	Boundary	Class Name	editRating
Description	This class displays an editable form where users can update a previously submitted rating. It pre-fills existing data, allows changes		

	and provides an option to save the updated rating.
--	--

Type of Class	Controller	Class Name	RatingController
Description	This controller handles all operations related to ratings. It manages adding, updating, retrieving and connects the UI with the data storage or backend service.		

Type of Class	Entity	Class Name	Rating
Description	This class defines the data model for a rating. It stores information such as rating ID, user ID, rating score, comment, and date. It is used to represent and process rating data in the system.		

3.3 Data Dictionary

3.3.1 User

Field Name	Description	Data Type	Constraint
UserID	Unique identifier for each user	INT	Primary Key
name	Full name of the user	VARCHAR(100)	NOT NULL
email	User's email address	VARCHAR(100)	NOT NULL, UNIQUE
phoneNum	User's phone number	VARCHAR(20)	NOT NULL, UNIQUE
ICNum	Identity Card number	VARCHAR(20)	NOT NULL, UNIQUE
address	Home address of the user	VARCHAR(255)	NOT NULL
gender	Gender of the user	VARCHAR(10)	NOT NULL
role	Role of the user	VARCHAR(20)	NOT NULL
password	User's hashed password	VARCHAR(255)	NOT NULL
profileImage	URL of user's profile picture	VARCHAR(255)	NOT NULL

3.3.2 Schedule

Field Name	Description	Data Type	Constraint
id	Unique identifier for schedule	String	PRIMARY KEY, NOT NULL
foremanName	Scheduled start Time	String	NOT NULL
date	Scheduled end Time	DATE	NOT NULL
startTime	Scheduled start Date	Timestamp	NOT NULL
endTime	Scheduled end Date	Timestamp	NOT NULL
vehicleName	Job status for the work	String	NOT NULL

vehicleColor	Flag for availability slots	String	NOT NULL
platNumber	When the work is recorded	String	NOT NULL
jobAssignment	When the work is updated	String	NOT NULL
status	Unique identifier for foreman	String	NULLABLE
completedAt	Unique identifier for workshop owner	TimeStamp	NULLABLE
notes	Vehicle name to be working on.	String	NULLABLE
photoUrls	Vehicle color to be working on.	String	NULLABLE
createdAt	Vehicle flat number to be working on.	TimeStamp	NULLABLE
updateAt	Description of vehicle, detailed work that need to be done	TimeStamp	NULLABLE

3.3.3 Inventory

Field Name	Description	Data Type	Constraint
id	Unique identifier for each inventory item	String	PRIMARY KEY, NOT NULL
brand	Brand of the inventory item	String	NOT NULL
model	Model name of the inventory item	String	NOT NULL
category	Type or category of the inventory item	String	NOT NULL

price	Price of the inventory item	Double	NOT NULL, > 0
quantity	Number of items available in stock	Integer	NOT NULL, >= 0
imageUrl	URL or path to the item's image	String	NOT NULL
id	Unique identifier for each inventory item	String	PRIMARY KEY, NOT NULL
brand	Brand of the inventory item	String	NOT NULL

3.3.4 Payment

Field Name	Description	Data Type	Constraint
paymentID	The unique identifier for the payment transaction.	INT(20)	PRIMARY KEY
paymentMethod	The method used for making the payment.	VARCHAR(50)	NOT NULL
paymentAmount	The total amount paid for the payment transaction.	DOUBLE	NOT NULL
paymentStatus	The status of the payment.	VARCHAR(50)	NOT NULL
paymentReference	A reference or remark for the payment (e.g., payment ID or note).	VARCHAR(50)	NOT NULL

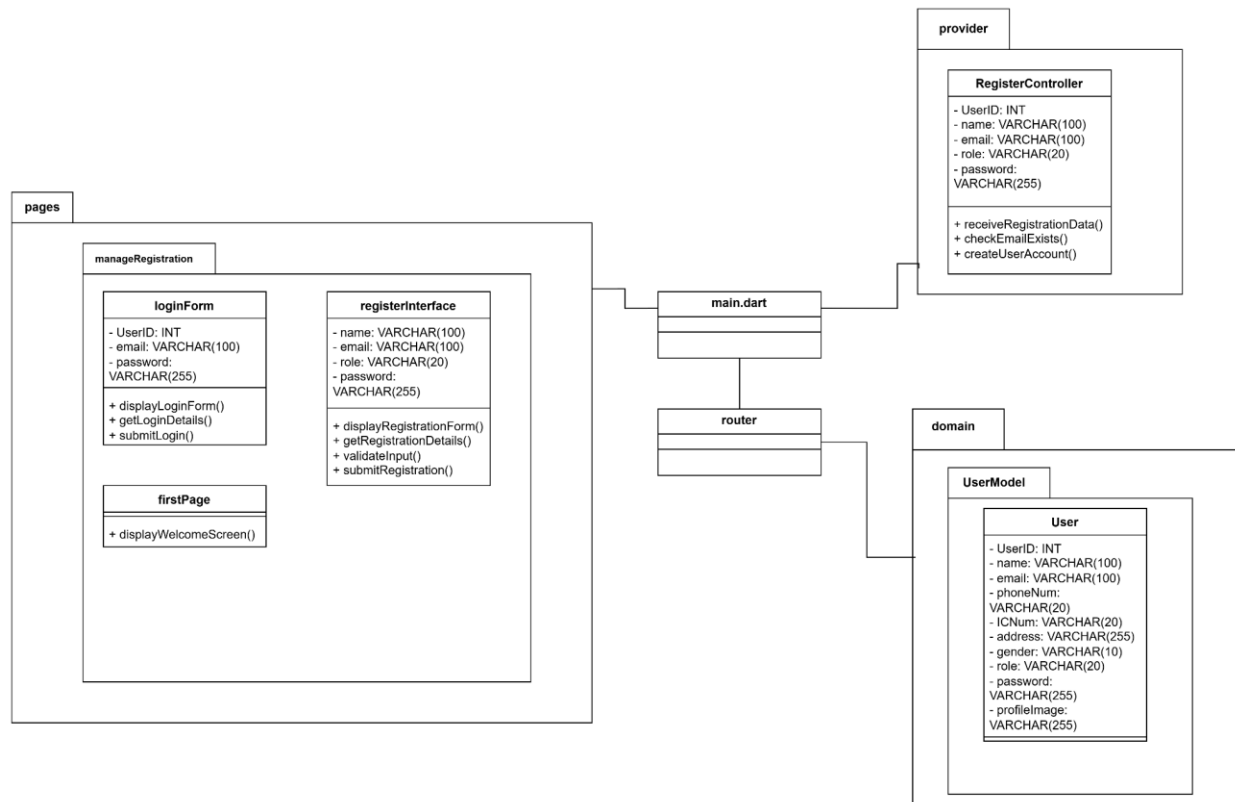
paymentDate	The date and time when the payment was made.	DATE	NOT NULL
foremanId	The unique identifier for the foreman who completed the task, foreign key referring to the User table.	INT(20)	FOREIGN KEY
workshopOwnerID	The unique identifier for the workshop owner, foreign key referring to the User table.	INT(20)	FOREIGN KEY

3.3.5 Rating

Field Name	Description	Data Type	Constraint
RatingID	Unique identifier for each rating	INT	Primary Key
RaterUserID	ID of the user who gives the rating	INT	Foreign Key → User(UserID)
RatedUserID	ID of the user who receives the rating	INT	Foreign Key → User(UserID)
ratingValue	Numeric score given by the user	INT	NOT NULL
comment	Comment about the rating	VARCHAR(255)	NOT NULL

4.0 DETAIL DESIGN

4.1 Manage Registration and User Profile



4.1.1 firstPage [FPS-PACK-101]

Class Type	Boundary Class	
Responsibility	This class shows the first screen of the app where users can choose to log in or register	
Attributes	Attributes Name	Attributes Types
	NOT APPLICABLE	NOT APPLICABLE
Methods	Method Name	Description

	displayWelcomeScreen()	Displays buttons/options for "Login" and "Register"
Algorithm	Function displayWelcomeScreen showAppLogo display "Login" button display "Register" button wait for user interaction If user clicks "Login" Call onLoginSelected Else if user clicks "Register" Call onRegisterSelected End Function	

4.1.2 loginForm [FPS-PACK-102]

Class Type	Boundary Class	
Responsibility	This class displays the login form where users enter their email and password.	
Attributes	Attributes Name	Attributes Types
	UserID	INT
	email	VARCHAR(100)
	password	VARCHAR(255)
Methods	Method Name	Description
	displayLoginForm()	Displays the form where users input their email and password.
	getLoginDetails()	Retrieves the email and password entered by the user.
	submitLogin()	Sends the entered credentials to the control class for authentication.
Algorithm	Function displayLoginForm Show login form with fields: email, password Wait for user input Call getLoginDetails End Function	

	Function getLoginDetails Retrieve 'email' from input field Retrieve 'password' from input field Call submitLogin(email, password) End Function Function submitLogin(email, password) Pass email and password to Control Class for authentication If authentication is successful Redirect to User Dashboard Else Display error message "Invalid email or password" End Function
--	--

4.1.3 registerInterface [FPS-PACK-103]

Class Type	Boundary Class	
Responsibility	This class provides the user interface for account registration. It includes form fields to collect user details such as name, email, password and role (Workshop Owner or Foreman), along with input validation and a sign-up button.	
Attributes	Attributes Name	Attributes Types
	name	VARCHAR(100)
	email	VARCHAR(100)
	role	VARCHAR(20)
	password	VARCHAR(255)
Methods	Method Name	Description
	displayRegistrationForm()	Displays the registration form with fields for name, email, role, and password.
	getRegistrationDetails()	Retrieves user input from the registration form.

	validateInput()	Validates the entered data
	submitRegistration()	Sends the registration data to the control class for processing.
Algorithm	Function displayRegistrationForm Show form fields: name, email, password, role (dropdown) Show "Sign Up" button Wait for user input Call getRegistrationDetails End Function Function getRegistrationDetails Retrieve 'name' from input field Retrieve 'email' from input field Retrieve 'password' from input field Retrieve 'role' from input field Call validateInput(name, email, password, role) End Function Function validateInput(name, email, password, role) If any field is empty Display error message "All fields are required" Else if email format is invalid Display error message "Invalid email format" Else Call submitRegistration(name, email, password, role) End Function Function submitRegistration(name, email, password, role) Pass registration data to Control Class for account creation If registration is successful Display message "Account created successfully" Redirect to login page Else Display error message "Registration failed, please try again" End Function	

4.1.4 RegisterController [FPS-PACK-701]

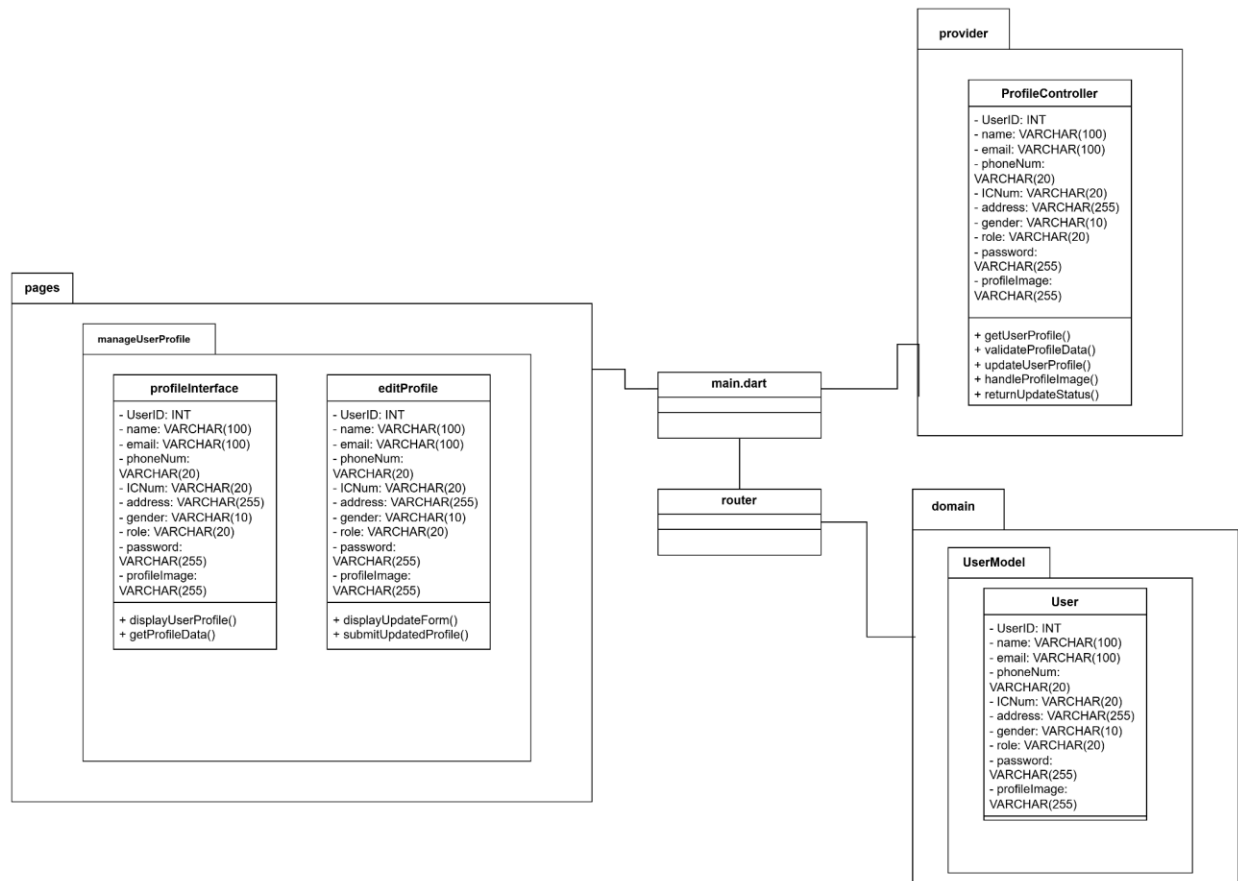
Class Type	Controller Class
Responsibility	This controller handles all actions related to user registration. It receives input from the registration interface, validates the data, interacts with the user entity or database to create new accounts, and returns appropriate success or error responses

Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	role	VARCHAR(20)
	password	VARCHAR(255)
Methods	Method Name	Description
	receiveRegistrationData()	Receives user input (name, email, password, role) from the boundary class.
	createUserAccount()	Stores the new user account in the database or entity.
	checkEmailExists()	Verifies if the provided email already exists in the database.
Algorithm	Function receiveRegistrationData(name, email, password, role) Call validateRegistrationData(name, email, password, role) End Function Function checkEmailExists(email) Query database for existing email If email exists Call returnResponse("Email already registered") Else Call createUserAccount(name, email, password, role) End Function Function createUserAccount(name, email, password, role) Hash the password Insert new user record into database with name, email, hashed password, and role If insertion successful Call returnResponse("Registration successful") Else Call returnResponse("Registration failed") End Function	

4.1.5 User [FPS-PACK-801]

Class Type	Entity Class	
Responsibility	This entity class represents the user data model. It stores user-related information such as name, email, phone number, password, and role (Workshop Owner or Foreman).	
Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	role	VARCHAR(20)
	password	VARCHAR(255)
Methods	Method Name	Description
	NOT APPLICABLE	NOT APPLICABLE
Algorithm	NOT APPLICABLE	

Manage User Profile



4.1.6 profileInterface [FPS-PACK-201]

Class Type	Boundary Class	
Responsibility	This class displays the user's profile information such as name, email, phone number, and role.	
Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	phoneNum	VARCHAR(20)
	ICNum	VARCHAR(20)

	address	VARCHAR(255)
	gender	VARCHAR(10)
	role	VARCHAR(20)
	password	VARCHAR(255)
	profileImage	VARCHAR(255)
Methods	Method Name	Description
	displayUserProfile()	Displays the user's profile information in a read-only format.
	getProfileData()	Retrieves user data from the controller or entity for display purposes.
Algorithm	Function displayUserProfile Call getProfileData Display the following fields in read-only mode: - Name - Email - Phone Number - IC Number - Address - Gender - Role - Profile Image End Function Function getProfileData Request user profile information from Controller Class Return data for display in the profile UI End Function	

4.1.7 editProfile [FPS-PACK-202]

Class Type	Boundary Class
Responsibility	This class provides the interface that allows users to update their profile information.

Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	phoneNum	VARCHAR(20)
	ICNum	VARCHAR(20)
	address	VARCHAR(255)
	gender	VARCHAR(10)
	role	VARCHAR(20)
	password	VARCHAR(255)
	profileImage	VARCHAR(255)
Methods	Method Name	Description
	displayUpdateForm()	Displays the user profile form with editable fields.
	submitUpdatedProfile()	Sends the updated data to the controller for validation and saving.
Algorithm	Function displayUpdateForm Call loadProfileData Show editable fields: - Name - Phone Number - Address - Gender - Profile Image Show "Save Changes" button End Function Function submitUpdatedProfile Collect all updated fields from form	

	Validate inputs (e.g., required fields, proper formats) Send data to Controller Class for processing If update is successful Call showMessage Else Call showErrorMessage End Function
--	---

4.1.8 ProfileController [FPS-PACK-702]

Class Type	Controller Class	
Responsibility	This controller manages profile-related operations such as retrieving user data, handling profile updates, validating changes, and saving updated information to the database.	
Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	phoneNum	VARCHAR(20)
	ICNum	VARCHAR(20)
	address	VARCHAR(255)
	gender	VARCHAR(10)
	role	VARCHAR(20)
	password	VARCHAR(255)
	profileImage	VARCHAR(255)
Methods	Method Name	Description

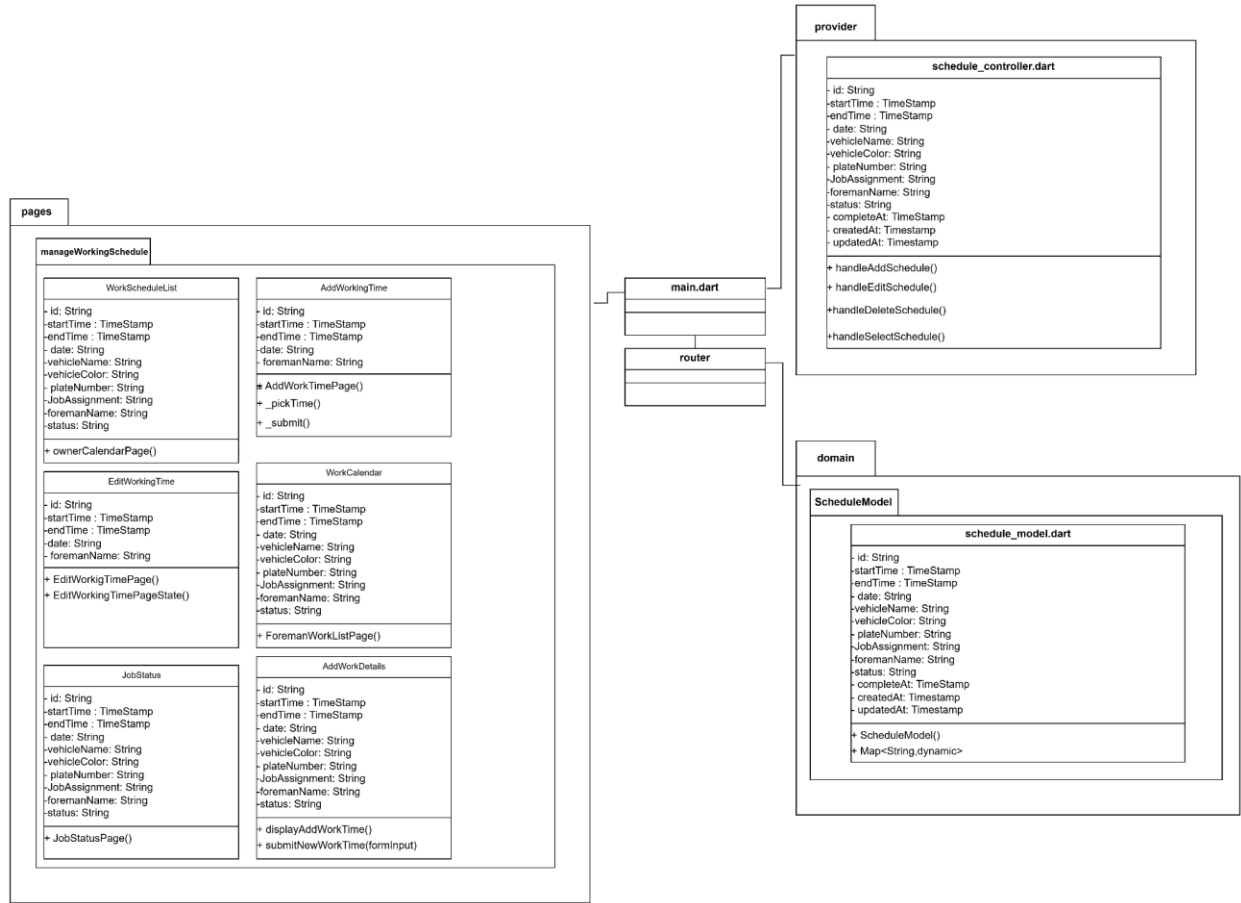
	getUserProfile(userID)	Retrieves user profile information from the database using the user ID.
	updateUserProfile()	Updates the user record in the database with the new profile information.
Algorithm	Function getUserProfile(userID) Query database for user data where UserID = userID Return user profile information to boundary class End Function Function updateUserProfile(userID, name, phoneNum, address, gender, profileImage) If profileImage is not null Call handleProfileImage(profileImage) Update user record in database with new data If update is successful Call returnUpdateStatus("Profile updated successfully") Else Call returnUpdateStatus("Profile update failed") End Function	

4.1.9 User [FPS-PACK-801]

Class Type	Entity Class	
Responsibility	This entity class represents the user data model. It stores user-related information such as name, email, phone number, password, and role (Workshop Owner or Foreman).	
Attributes	Attributes Name	Attributes Types
	UserID	INT
	name	VARCHAR(100)
	email	VARCHAR(100)
	phoneNum	VARCHAR(20)
	ICNum	VARCHAR(20)

	address	VARCHAR(255)
	gender	VARCHAR(10)
	role	VARCHAR(20)
	password	VARCHAR(255)
	profileImage	VARCHAR(255)
Methods	Method Name	Description
	NOT APPLICABLE	NOT APPLICABLE
Algorithm	NOT APPLICABLE	

4.2 Manage Working Schedule



4.2.1 WorkScheduleList [FPS-PACK-301]

Class Type	Boundary	
Responsibility	Display all the available foreman to the workshop owner.	
Attributes	Attributes Name	Attributes Type
	id	String
	startTime	Timestamp
	endTime	Timestamp
	date	Timestamp
	vehicleName	String
	vehicleColor	String
	vehicleNo	String
	jobAssignment	String
	foremanName	String
	status	String
Methods	Method Name	Description
	ownerCalendarPage()	Display foreman list with all details using calendar.
Algorithm	ownerCalendarPage() START	

	Receive schedules list FOR each schedule IN schedules DO Format schedule details Display schedule in the UI END
--	---

4.2.2 Add working time [FPS-PACK-302]

Class Type	Boundary	
Responsibility	Get all the work time with the details from the foreman into the system.	
Attributes	Attributes Name	Attributes Type
	id	String
	foremanName	String
	startTime	Timestamp
	endTime	Timestamp
	date	String
Methods	Method Name	Description
	AddWorkTimePage()	Display the empty form of add working time.
	_pickTime()	Pick time availability.
	_submit()	Submit the work time to be created.
Algorithm	AddWorkTimePage() START Set isAddMode = FALSE	

	Initialize formData with default values (empty strings, 0 wages) Render empty form fields for input END _pick() START Check workShift is not empty Check date is valid Check dailyWages > 0 IF validation fails THEN Call showValidationErrors() EXIT Call controller.handleAddSchedule(formInput) END _submit() START Check workShift is not empty Check date is valid Check dailyWages > 0 IF validation fails THEN Call showValidationErrors() EXIT Call controller.handleAddSchedule(formInput) END
--	---

4.2.3 Edit Working Time [FPS-PACK-303]

Class Type	Boundary	
Responsibility	Update all the work time with the details from the foreman into the system.	
Attributes	Attributes Name	Attributes Type
	id	String
	startTime	Timestamp
	endTime	Timestamp

	date	String
Methods	Method Name	Description
	EditWorkingTimePage()	Shows the form with existing schedule data for editing
	EditWorkingTimePageState()	Consists of fetch data, update and delete availability in the system.
Algorithm	EditWorkingTimePage() START Set isEditMode = TRUE Set formData = schedule Pre-fill form fields with formData values END EditWorkingTimePageState() START Check updated workShift, date, and wages IF validation fails THEN Call showValidationErrors() EXIT Call controller.handleEditSchedule(scheduleId, updatedData) END	

4.2.4 WorkCalendar [FPS-PACK-304]

Class Type	Boundary	
Responsibility	Display all the work list that assigned to user.	
Attributes	Attributes Name	Attributes Type
	id	String

	startTime	Timestamp
	endTime	Timestamp
	startDate	Timestamp
	vehicleName	String
	vehicleColor	String
	vehicleNo	String
	jobDescription	String
	foremanName	String
	status	String
Methods	Method Name	Description
	ForemanWorkListPage()	Display work list with all details.
Algorithm	ForemanWorkListPage() START Receive schedules list FOR each schedule IN schedules DO Format schedule details Display schedule in the UI END	

4.2.5 JobStatus [FPS-PACK-305]

Class Type	Boundary
------------	----------

Responsibility	Display all job details and user input whether the job is complete or not.	
Attributes	Attributes Name	Attributes Type
	id	String
	vehicleName	String
	vehicleColor	String
	plateNumber	String
	jobAssignment	String
	date	String
	startTime	Timestamp
	endTime	Timestamp
Methods	Method Name	Description
	JobStatusPage()	Shows the form with existing job status data for display
	JobStatusPageState()	Consists of fetching data and updating job status in the system.
Algorithm	JobStatusPage() START Set isEditMode = FALSE Initialize formData with default values (empty strings, 0 wages) Render empty form fields for input END JobStatusPageState() START	

	Check workShift is not empty Check date is valid IF validation fails THEN Call showValidationErrors() EXIT Call controller.handleAddSchedule(formInput) END
--	---

4.2.6 AddWorkDetails [FPS-PACK-306]

Class Type	Boundary	
Responsibility	Get all the work time with the details from the foreman into the system.	
Attributes	Attributes Name	Attributes Type
	id	String
	foremanName	String
	startTime	Timestamp
	endTime	Timestamp
	date	String
	vehicleName	String
	vehicleColor	String
	vehicleNo	String
	jobDescription	String
Methods	Method Name	Description
	AddWorkDetailsPage()	Shows the empty form to be input for work details.

	AddWorkDetailsPageState()	Consists of getting data and submitting work details in the database system.
Algorithm	AddWorkDetailsPage() START Set isEditMode = FALSE Initialize formData with default values (empty strings, 0 wages) Render empty form fields for input END AddWorkDetailsPageState() START Check workShift is not empty Check date is valid IF validation fails THEN Call showValidationErrors() EXIT Call controller.handleAddSchedule(formInput) END	

4.2.7 ScheduleController [FPS-PACK-307]

Class Type	Controller	
Responsibility	Handles flow between view and model, processes user actions for managing schedules.	
Attributes	Attributes Name	
	id	String
	startTime	Timestamp
	endTime	Timestamp
	date	Timestamp
	vehicleName	String

	vehicleColor	String
	vehicleNo	String
	jobDescription	String
	foremanName	String
	status	String
	completedAt	Timestamp
	createdAt	Timestamp
	updatedAt	Timestamp
Methods	Method Name	
	handleAddSchedule(newSchedule)	Handles submission of new schedule data.
	handleEditSchedule(scheduleId, updatedSchedule)	Handles editing and saving changes to a schedule.
	handleDeleteSchedule(scheduleId)	Handles deletion of schedule.
	handleSelectSchedule(scheduleId, foremanId)	Handles selection of available schedules.
Algorithm	handleAddSchedule(newSchedule) START Call workingScheduleModel.createSchedule(newSchedule) IF result = success THEN schedulingInterface.notifyActionResult(TRUE, "add") ELSE schedulingInterface.showErrorMessage("Schedule creation failed") END	

	handleEditSchedule(scheduleId, updatedSchedule) START Validate updatedSchedule (check workShift, date, wages) Call workingScheduleModel.updateSchedule(scheduleId, updatedSchedule) IF result = success THEN schedulingInterface.notifyActionResult(TRUE, "edit") ELSE schedulingInterface.showErrorMessage("Schedule update failed") END handleDeleteSchedule(scheduleId) START Confirm delete via UI Call workingScheduleModel.deleteSchedule(scheduleId) IF result = success THEN schedulingInterface.notifyActionResult(TRUE, "delete") ELSE schedulingInterface.showErrorMessage("Delete failed") END handleSelectSchedule(scheduleId, foremanId) START Call workingScheduleModel.selectSchedule(scheduleId, foremanId) IF result = success THEN schedulingInterface.notifyActionResult(TRUE, "select") ELSE schedulingInterface.showErrorMessage("Selection failed") END
--	---

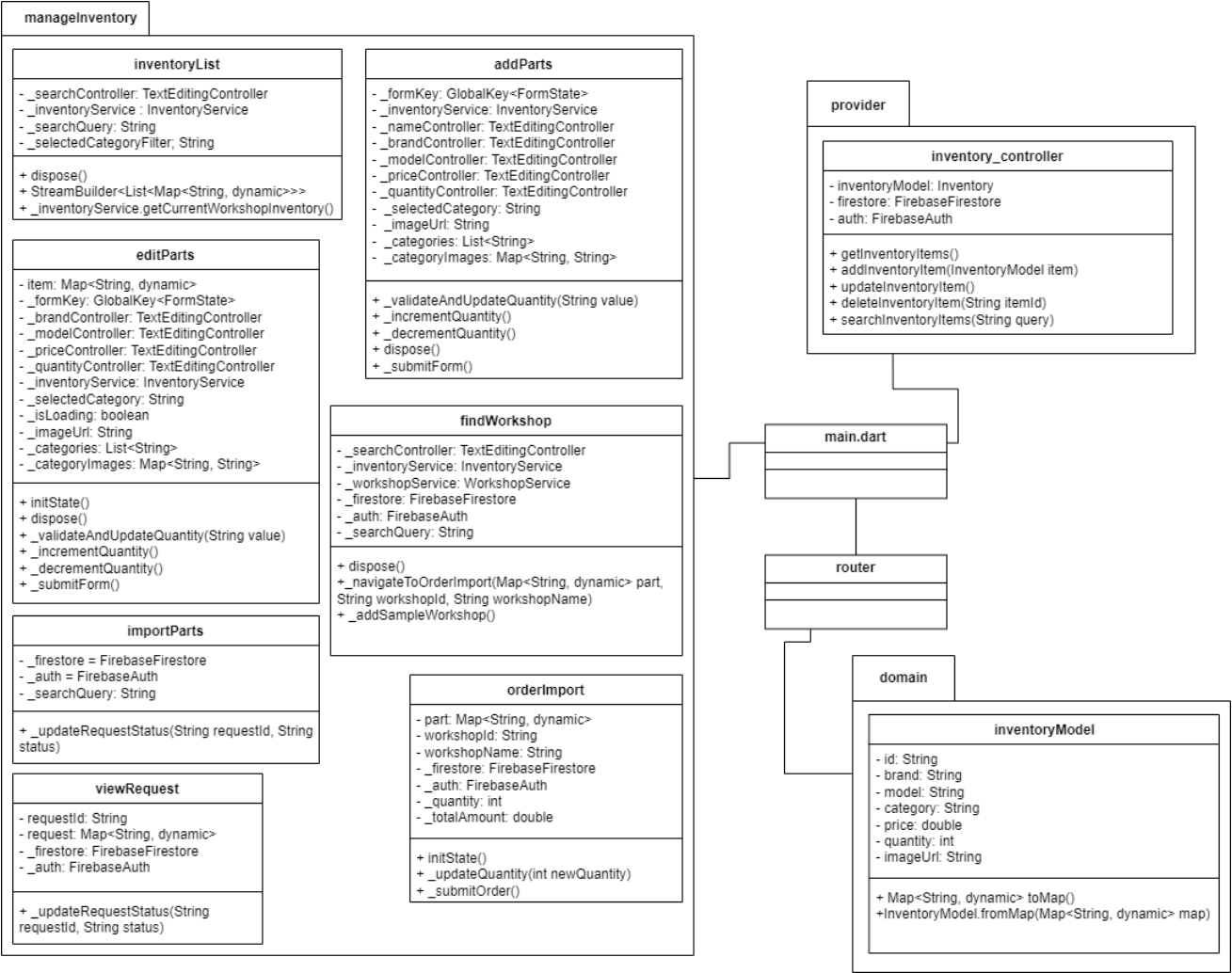
4.2.8 ScheduleModel [FPS-PACK-308]

Class Type	Entity	
Responsibility	Holds all the data in the manage working schedules.	
Attributes	Attributes Name	

	scheduleID	INT
	startTime	Timestamp
	endTime	Timestamp
	startDate	Timestamp
	endDate	Timestamp
	status	String
	isForemanAvailable	Boolean
	createdAt	Timestamp
	updatedAt	Timestamp
	foremanID	INT
	ownerID	INT
	vehicleName	String
	vehicleColor	String
	vehicleNo	String
Methods	Method Name	
	ScheduleModel()	Consists of all data to create schedule model
	Map<String, dynamic> toMap()	Converts item into map for database saving
Algorithm	ScheduleModel() START Create map with item properties Return map object	

	END Map<String, dynamic> toMap() START Create new ScheduleModel instance Set properties from map values Return ScheduleModel object END
--	--

4.3 Manage Inventory



4.3.1 inventoryList [FPS-PACK-401]

Class Type	Boundary	
Responsibility	This class displays the main inventory dashboard for workshop owners. It shows a list of available inventory items, including their category-based image, brand, model, quantity, and price. It serves as the primary interface for users to search for items, navigate to edit item details, and trigger item deletion.	
Attributes	Attributes Name	Attributes Type
	_searchController	TextEditingController
	_inventoryService	InventoryService
	_searchQuery	String
	_selectedCategoryFilter	String
Methods	Method Name	Description
	dispose()	Disposes of controller and service on teardown.
	StreamBuilder<List<Map<String, dynamic>>>	Listens to real-time inventory updates.
	_inventoryService.getCurrentWorkshopInventory()	Fetches current inventory items for workshop.
Algorithm	dispose() START Dispose _searchController Dispose _inventoryService END StreamBuilder<List<Map<String, dynamic>>> START Listen to _inventoryService.getCurrentWorkshopInventory() IF data exists THEN Filter items based on _searchQuery and _selectedCategoryFilter Display filtered items in list	

	<pre> ELSE Show loading indicator END IF END getCurrentWorkshopInventory() START Get current user's workshop ID Query Firestore for items where workshopId matches Return stream of items END </pre>
--	---

4.3.2 addParts [FPS-PACK-402]

Class Type	Boundary	
Responsibility	Handles the user interface for adding a new inventory item. It includes input fields for brand, model, category, quantity, and price. It facilitates category-based image display and includes client-side validation, showing success or error messages to the user.	
Attributes	Attributes Name	Attributes Type
	_formKey	GlobalKey
	_inventoryService	InventoryService
	_nameController	TextEditingController
	_brandController	TextEditingController
	_modelController	TextEditingController
	_priceController	TextEditingController
	_quantityController	TextEditingController
	_selectedCategory	String

	_imageUrl	String
	_categories	List
	_categoryImages	Map<String, String>
Methods	Method Name	Description
	_validateAndUpdateQuantity(String value)	Validates and updates quantity input.
	_incrementQuantity()	Increments item quantity.
	_decrementQuantity()	Decrements item quantity.
	dispose()	Disposes controllers and form.
	_submitForm()	Validates and submits item data.
Algorithm	_validateAndUpdateQuantity(String value) START IF value is empty THEN Set quantity to 1 ELSE Parse value to integer IF parsed value < 1 THEN Set quantity to 1 ELSE Update quantity with parsed value END IF END IF END _incrementQuantity() START Get current quantity Increment by 1 Update _quantityController END _decrementQuantity() START Get current quantity	

	<pre> IF quantity > 1 THEN Decrement by 1 Update _quantityController END IF END dispose() START Dispose all controllers Dispose form key END _submitForm() START IF form is valid THEN Create new inventory item with form data Add item to inventory service Show success message Navigate back to inventory list ELSE Show error message END IF END </pre>
--	---

4.3.3 editParts [FPS-PACK-403]

Class Type	Boundary	
Responsibility	Provides a UI for updating an existing inventory item's details. It includes fields to update brand, model, quantity, price, category, and image. Existing item data is pre-filled and validated before being updated.	
Attributes	Attributes Name	Attributes Type
	item	Map<String, dynamic>
	_formKey	GlobalKey
	_brandController	TextEditingController

	_modelController	TextEditingController
	_priceController	TextEditingController
	_quantityController	TextEditingController
	_inventoryService	InventoryService
	_selectedCategory	String
	_isLoading	boolean
	_imageUrl	String
	_categories	List
	_categoryImages	Map<String, String>
Methods	Method Name	Description
	initState()	Initializes form with existing item data.
	dispose()	Cleans up controllers and state.
	_validateAndUpdateQuantity(value)	Validates and updates quantity field.
	_incrementQuantity()	Increases the quantity field.
	_decrementQuantity()	Decreases the quantity field.
	_submitForm()	Validates input and updates item in database.
Algorithm	initState() START Initialize all controllers with existing item data Set _selectedCategory to item's category	

```
    Set _imageUrl to item's image  
END
```

dispose()

```
START  
    Dispose all controllers  
    Dispose form key  
END
```

_validateAndUpdateQuantity(value)

```
START  
    IF value is empty THEN  
        Set quantity to 1  
    ELSE  
        Parse value to integer  
        IF parsed value < 1 THEN  
            Set quantity to 1  
        ELSE  
            Update quantity with parsed value  
        END IF  
    END IF  
END
```

_incrementQuantity()

```
START  
    Get current quantity  
    Increment by 1  
    Update _quantityController  
END
```

_decrementQuantity()

```
START  
    Get current quantity  
    IF quantity > 1 THEN  
        Decrement by 1  
        Update _quantityController  
    END IF  
END
```

_submitForm()

```
START  
    IF form is valid THEN  
        Update item with new form data  
        Update item in inventory service  
        Show success message  
        Navigate back to inventory list
```

	ELSE Show error message END IF END
--	---

4.3.4 importParts [FPS-PACK-404]

Class Type	Boundary	
Responsibility	This class manages the list of incoming and outgoing part requests between workshops. It displays request status, item details, and the date and time of requests.	
Attributes	Method Name	Description
	_firestore	FirebaseFirestore
	_auth	FirebaseAuth
	_searchQuery	String
Methods	Method Name	Description
	_updateRequestStatus(String requestId, String status)	Updates the status of a part request.
Algorithm	_updateRequestStatus(String requestId, String status) START Get current user's workshop ID Update request document in Firestore IF status is 'approved' THEN Update inventory quantities Show success message ELSE IF status is 'cancelled' THEN Show cancellation message END IF END	

4.3.5 findWorkshop [FPS-PACK-405]

Class Type	Boundary	
Responsibility	Provides a search interface for parts and locating other workshops within the marketplace. Shows available parts, workshop info, and allows navigation to place part orders.	
Attributes	Attributes Name	Attributes Type
	_searchController	TextEditingController
	_inventoryService	InventoryService
	_workshopService	WorkshopService
	_firestore	FirebaseFirestore
	_auth	FirebaseAuth
	_searchQuery	String
Methods	Method Name	Description
	dispose()	Disposes all controllers.
	_navigateToOrderImport(Map<String, dynamic> part, String workshopId, String workshopName)	Navigates to order page with part and workshop info.
	_addSampleWorkshop()	Adds a mock workshop to Firestore for testing.
Algorithm	dispose() START Dispose _searchController Dispose _inventoryService Dispose _workshopService END _navigateToOrderImport(Map<String, dynamic> part, String workshopId, String workshopName) START	

	Navigate to orderImport page Pass part details, workshopId, and workshopName END _addSampleWorkshop() START Create sample workshop data Add to Firestore collection Show success message END
--	---

4.3.6 orderImport **[FPS-PACK-406]**

Class Type	Boundary	
Responsibility	Displays the part details from another workshop and allows the owner to select quantity and place an order. Shows part info, available quantity, price, and total cost. Validates quantity and displays success or error	
Attributes	Attributes Name	Attributes Type
	part	Map<String, dynamic>
	workshopId	String
	workshopName	String
	_firestore	FirebaseFirestore
	_auth	FirebaseAuth
	_quantity	int
	_totalAmount	double
Methods	Method Name	Description
	initState()	Initializes request page with part info.

	<code>_updateQuantity()</code>	Changes the selected quantity.
	<code>_submitOrder()</code>	Submits order to Firestore.
Algorithm	<code>initState()</code> START Initialize part details from arguments Set initial quantity to 1 Calculate initial total amount END <code>_updateQuantity()</code> START IF new quantity is valid THEN Update <code>_quantity</code> Calculate new total amount Update UI ELSE Show error message END IF END <code>_submitOrder()</code> START Get current user's workshop ID Create order document in Firestore IF order creation successful THEN Show success message Navigate back to <code>findWorkshop</code> ELSE Show error message END IF END	

4.3.7 viewRequest [FPS-PACK-407]

Class Type	Boundary
Responsibility	Displays detailed information about a specific request. Shows request status, part info, request date and time, and options to approve or cancel the request.

Attributes	Attributes Name	Attributes Type
	requestId	String
	request	Map<String, dynamic>
	_firestore	FirebaseFirestore
	_auth	FirebaseAuth
Methods	Method Name	Description
	_updateRequestStatus(String requestId, String status)	Updates request status (approve, cancel)
Algorithm	_updateRequestStatus(String requestId, String status) START Get current user's workshop ID Update request document in Firestore IF status is 'approved' THEN Update inventory quantities Show success message ELSE IF status is 'cancelled' THEN Show cancellation message END IF Navigate back to importParts END	

4.3.7 inventory_controller [FPS-PACK-408]

Class Type	Controller	
Responsibility	Handles inventory management logic such as CRUD operations. Integrates with Firebase and supports search functionality, user auth, and error handling.	
Attributes	Attributes Name	Attributes Type
	_firestore	FirebaseFirestore

	_auth	FirebaseAuth
Methods	Method Name	Description
	getInventoryItems()	Fetches inventory items from database
	addInventoryItem(Inventory Model item)	Adds a new item to Firestore
	updateInventoryItem()	Updates existing inventory item
	deleteInventoryItem(String itemId)	Deletes inventory item by ID
	searchInventoryItems(String query)	Searches inventory records
Algorithm	getInventoryItems() START Get current user's workshop ID Query Firestore for items Return list of InventoryModel objects END addInventoryItem(InventoryModel item) START Get current user's workshop ID Add item to Firestore Return success status END updateInventoryItem() START Get current user's workshop ID Update item in Firestore Return success status END deleteInventoryItem(String itemId) START Get current user's workshop ID Delete item from Firestore Return success status END	

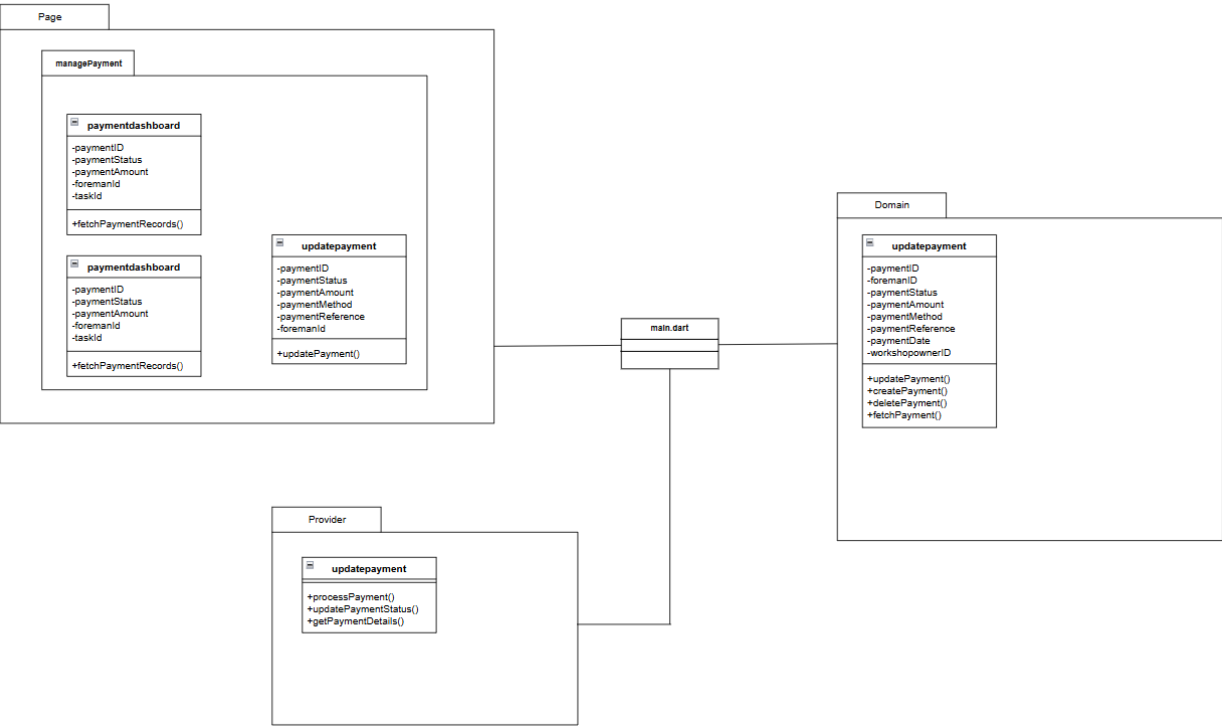
	searchInventoryItems(String query) START Get current user's workshop ID Query Firestore with search parameters Return filtered list of InventoryModel objects END
--	---

4.3.8 inventoryModel [FPS-PACK-409]

Class Type	Entity	
Responsibility	Defines the structure of an inventory item used for database mapping and UI data transfer.	
Attributes	Attributes Name	Attributes Type
	id	String
	brand	String
	model	String
	category	String
	price	double
	quantity	int
	imageUrl	String
Methods	Method Name	Description
	Map<String, dynamic> toMap()	Converts item into map for database saving
	InventoryModel.fromMap(Map<String, dynamic>)	Creates item object from retrieved data
Algorithm	toMap()	

	START Create map with item properties Return map object END InventoryModel.fromMap(Map<String, dynamic>) START Create new InventoryModel instance Set properties from map values Return InventoryModel object END
--	--

4.4 Manage Payment



4.4.1 paymentdashboard [FPS-PACK-501]

Class Type	Boundary Class
------------	----------------

Responsibility	This class represents the payment dashboard view for workshop owners and foremen. It displays payment details such as payment ID, amount, status, task information, and date for each payment.	
Attributes	Attributes Name	Attributes Type
	paymentID	INT
	paymentStatus	VARCHAR
	paymentAmount	DOUBLE
	foremanId	INT
	time	INT
Methods	Method Name	Description
	fetchPaymentRecords()	To fetch all payment records and display them on the dashboard.
Algorithm	START 1. Connect to the database. 2. Retrieve data from the paymentController. 3. Display the payment records on the dashboard page. End	

4.4.2 updatepayment [FPS-PACK-502]

Class Type	Boundary Class	
Responsibility	This class handles the user interface for entering payment details. Workshop owners can input payment method, amount, and other transaction-related information.	
Attributes	Attributes Name	Attributes Type

	paymentID	INT
	paymentMethod	VARCHAR(50)
	paymentAmount	DOUBLE
	time	INT
	paymentReference	VARCHAR(50)
	paymentStatus	VARCHAR(50)
	foremanId	INT
Methods	Method Name	Description
	updatePayment()	Update selected payment.
Algorithm	Start 1. Display the payment form. 2. Validate the entered data. 3. If data is valid, submit it for processing. End	

4.4.3 addpayment [FPS-PACK-503]

Class Type	Boundary Class	
Responsibility	This class displays the payment transaction history for workshop owners. It shows the list of completed and pending payments, filtered by date or task.	
Attributes	Attributes Name	Attributes Type
	paymentId	INT
	paymentStatus	VARCHAR(50)

	paymentAmount	DOUBLE
	paymentMethod	VARCHAR(50)
	time	INT
	foremanId	INT
	paymentDescription	VARCHAR(50)
	paymentReference	VARCHAR(50)
Methods	Method Name	Description
	fetchTransactionHistory()	To fetch and display the transaction history based on date or task.
Algorithm	START 1. Connect to the database. 2. Retrieve payment history data from the paymentController. 3. Display the payment history. END	

4.4.4 paymentController [FPS-PACK-504]

Class Type	Controller	
Responsibility	Handles the processing, updating, deleting, and listing of payment records between UI and data.	
Attributes	Method Name	Description

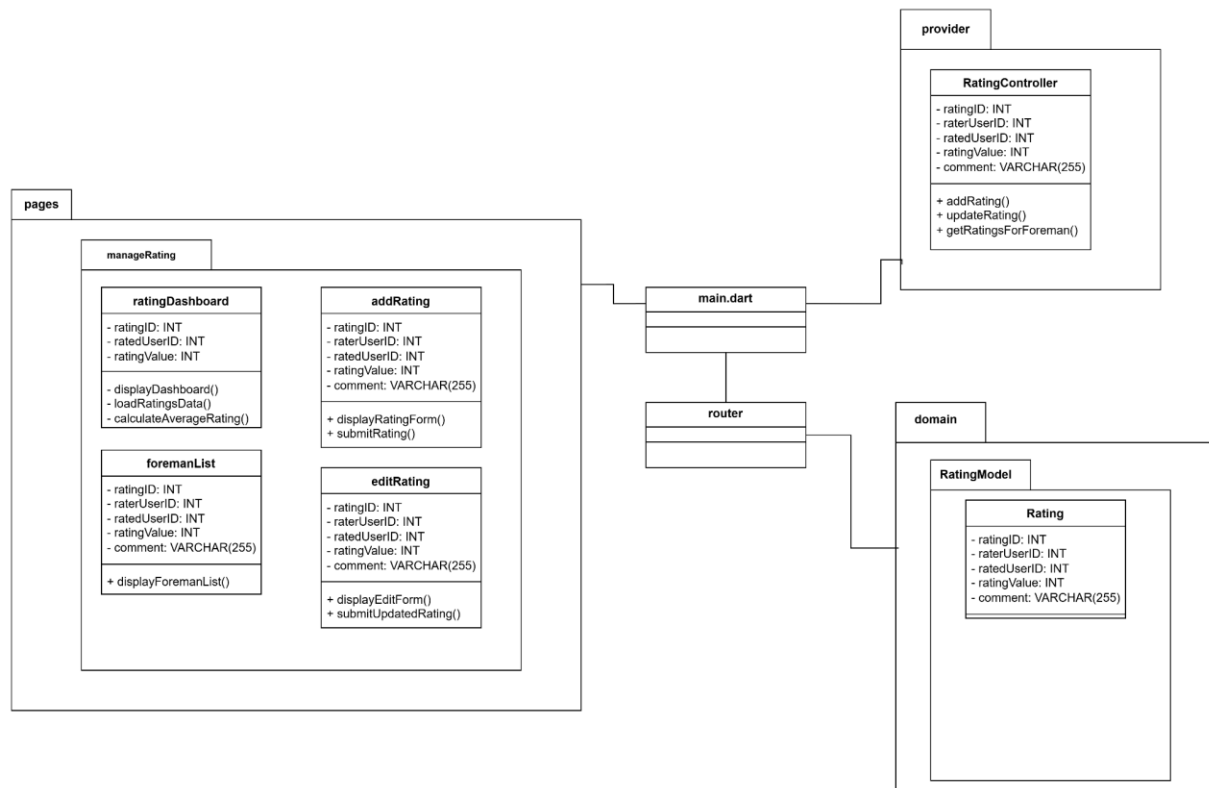
	NOT APPLICABLE	NOT APPLICABLE
Methods	Method Name	Description
	processPayment()	To process the payment and update the payment status.
	updatePaymentStatus()	To update the payment status (completed, pending, failed).
	getPaymentDetails()	To retrieve the details of a specific payment.
Algorithm	Start 1. Receive payment data from the paymentFormView. 2. Validate payment details. 3. Process the payment and update the status. 4. Notify the appropriate view with the payment status. End	

4.4.5 Payment [FPS-PACK-505]

Class Type	Entity	
Responsibility	Manages the storage and integrity of payment data, including foreign keys and timestamps.	
Attributes	Attributes Name	Attributes Type
	paymentId	INT
	foremanId	INT

	paymentReference	VARCHAR(50)
	paymentAmount	DOUBLE
	paymentMethod	VARCHAR(50)
	paymentStatus	VARCHAR(50)
	ownerID	INT
	Start time	INT
	End Time	INT
	payment_date	DATE
Methods	Method Name	Description
	createpayment()	Inserts a new payment into the database.
	updatePayment()	Updates an existing payment record by payment ID.
	fetchPayment()	Fetch payment by payment ID.
	deletePayment()	Deletes a payment record by payment ID.
Algorithm	START 1. Validate all required fields (Ensure fields). 2. Insert the validated payment data into the Payment table. 3. Return a success confirmation message to indicate the payment record was created. END	

4.5 Manage Rating



4.5.1 ratingDashboard [FPS-PACK-601]

Class Type	Boundary Class	
Responsibility	This class displays a dashboard view of all ratings submitted by the workshop owner. It shows an overview of rated foremen	
Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RatedUserID	INT
	ratingValue	INT
Methods	Method Name	Description

	displayDashboard()	Loads and shows a list or summary view of ratings submitted by the workshop owner
	loadRatingsData()	Retrieves all ratings associated with the logged-in workshop owner
	calculateAverageRating(foremanID: INT)	Computes and displays the average rating for each foreman
Algorithm	Function displayDashboard Call loadRatingsData For each rating in ratingsList Display RatedUserID and ratingValue Call calculateAverageRating(RatedUserID) End For End Function Function loadRatingsData Retrieve all ratings where RaterUserID = LoggedInUserID Store results in ratingsList End Function Function calculateAverageRating(foremanID) Retrieve all ratingValues where RatedUserID = foremanID Calculate average Display average rating End Function	

4.5.2 foremanList [FPS-PACK-602]

Class Type	Boundary Class	
Responsibility	This class displays a list of foremen that users can rate and edit. It includes basic foreman details such as name and profile to help users choose whom to rate.	
Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RaterUserID	INT

	RatedUserID	INT
	ratingValue	INT
	comment	VARCHAR(255)
Methods	Method Name	Description
	displayForemanList()	Loads and shows a list of foremen available to be rated
Algorithm	Function displayForemanList Retrieve all users with role = 'Foreman' For each foreman in list Display foreman name and profile image Provide "Rate" and "Edit Rating" options End For End Function	

4.5.3 addRating [FPS-PACK-603]

Class Type	Boundary Class	
Responsibility	This class provides the interface for users to submit a new rating for a foreman. It includes input fields for rating score, comments and submission buttons	
Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RaterUserID	INT
	RatedUserID	INT
	ratingValue	INT
	comment	VARCHAR(255)
Methods	Method Name	Description

	displayRatingForm()	Loads and shows the form with input fields for rating and comment
	submitRating()	Sends the rating data to the controller or database for processing
Algorithm	Function displayRatingForm Show input field for ratingValue (1 to 5) Show textarea for comment Show "Submit" and "Cancel" buttons End Function Function submitRating Call validateInputs If inputs are valid Send RatingID, RaterUserID, RatedUserID, ratingValue, comment to controller Call showConfirmationMessage Call clearForm Else Call showErrorMessage End If End Function	

4.5.4 editRating [FPS-PACK-604]

Class Type	Boundary Class	
Responsibility	This class displays an editable form where users can update a previously submitted rating. It pre-fills existing data, allows changes and provides an option to save the updated rating.	
Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RaterUserID	INT
	RatedUserID	INT
	ratingValue	INT

	comment	VARCHAR(255)
Methods	Method Name	Description
	displayEditForm(ratingID: INT)	Loads the existing rating details and pre-fills the form
	submitUpdatedRating()	Sends the updated data to the controller to update the database
Algorithm	Function displayEditForm(ratingID) Retrieve rating record by ratingID Pre-fill form with ratingValue and comment Display form with "Update" and "Cancel" buttons End Function Function submitUpdatedRating Call validateInputs If inputs are valid Update rating record in database Call showSuccessMessage Else Call showErrorMessage End If End Function	

4.5.5 RatingController [FPS-PACK-706]

Class Type	Controller Class	
Responsibility	This controller handles all operations related to ratings. It manages adding, updating, retrieving and connects the UI with the data storage or backend service	
Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RaterUserID	INT
	RatedUserID	INT

	ratingValue	INT
	comment	VARCHAR(255)
Methods	Method Name	Description
	addRating()	Adds a new rating record to the database
	updateRating()	Updates an existing rating record with new values
	getRatingsForForeman()	Retrieves all ratings received by a specific foreman
Algorithm	Function addRating(raterUserID, ratedUserID, ratingValue, comment) If validateRatingInput(ratingValue, comment) is valid Insert new rating into database with raterUserID, ratedUserID, ratingValue, comment Return success response Else Return validation error End Function Function updateRating(ratingID, ratingValue, comment) If validateRatingInput(ratingValue, comment) is valid Update rating where RatingID = ratingID with new ratingValue and comment Return success response Else Return validation error End Function Function getRatingsForForeman(foremanID) Retrieve all ratings from database where RatedUserID = foremanID Return list of ratings End Function	

4.5.6 Rating [FPS-PACK-805]

Class Type	Entity Class
Responsibility	This class defines the data model for a rating. It stores information such as rating ID, user ID, rating score, comment, and date. It is used to represent and process rating data in the system.

Attributes	Attributes Name	Attributes Types
	RatingID	INT
	RaterUserID	INT
	RatedUserID	INT
	ratingValue	INT
	comment	VARCHAR(255)
Methods	Method Name	Description
	NOT APPLICABLE	NOT APPLICABLE
Algorithm	NOT APPLICABLE	

5.0 REQUIREMENT TRACEABILITY

Class ID	Class Name	Description	Dependency Package ID
[FPS-PACK-101]	firstPage	shows the first screen of the app where users can choose to log in or register	[FPS-PACK-100]
[FPS-PACK-102]	loginForm	shows the login form where users enter their email and password to log into the app	[FPS-PACK-100]
[FPS-PACK-103]	registerInterface	Shows the registration form where new users enter their details to create an account	[FPS-PACK-100]
[FPS-PACK-201]	profileInterface	Shows the user profile with details.It also lets users go to the edit page	[FPS-PACK-200]
[FPS-PACK-202]	editProfile	Display a form where users can edit and save their profile information	[FPS-PACK-200]
[FPS-PACK-301]	WorkScheduleList	Display all work list with all the work details.	[FPS-PACK-300]
[FPS-PACK-302]	AddWorkingTime	Shows user the form to add working time availability.	[FPS-PACK-300]
[FPS-PACK-303]	EditWorkingTime	Shows user the form to update working time availability.	[FPS-PACK-300]
[FPS-PACK-304]	WorkCalendar	Display working schedule in calendar form.	[FPS-PACK-300]

[FPS-PACK-305]	JobStatus	Shows user the form to update job status accordingly.	[FPS-PACK-300]
[FPS-PACK-306]	AddWorkDetails	Shows the user the form to add working details for a work.	[FPS-PACK-300]
[FPS-PACK-307]	ScheduleController	Handles all the data flow in working schedule management.	[FPS-PACK-300]
[FPS-PACK-308]	ScheduleModel	Holds all the data in working schedule module.	[FPS-PACK-300]
[FPS-PACK-401]	inventoryList	Displays all inventory items, supports filtering, and navigation to add, import and edit.	[FPS-PACK-400]
[FPS-PACK-402]	addParts	Provides a form for users to add new inventory items with validation.	[FPS-PACK-400]
[FPS-PACK-403]	editParts	Allows users to edit existing inventory item details with input validation.	[FPS-PACK-400]
[FPS-PACK-404]	importParts	Manages creation and tracking of part requests between workshops.	[FPS-PACK-400]
[FPS-PACK-405]	findWorkshop	Displays available items from other workshops and supports sending requests.	[FPS-PACK-400]
[FPS-PACK-406]	orderImport	Displays selected part details and lets owners submit orders to other workshops.	
[FPS-PACK-407]	viewRequest	Displays request details with options to	[FPS-PACK-400]

		approve or cancel the request.	
[FPS-PACK-408]	inventory_controller	Controls inventory operations including add, edit, delete, search, and request handling.	[FPS-PACK-400]
[FPS-PACK-409]	inventoryModel	Represents inventory item data structure and handles conversion between app and database.	[FPS-PACK-400]
[FPS-PACK-501]	PaymentDashboard	This class represents the view of payment dashboard.	[FPS-PACK-500]
[FPS-PACK-502]	addpayment	Lets the workshop owner easily add a new payment by entering all the necessary details like the amount, payment method, and reference number.	[FPS-PACK-500]
[FPS-PACK-503]	Updatepayment	Allows the workshop owner to update an existing payment, including changes to the amount, payment method, or any remarks.	[FPS-PACK-500]
[FPS-PACK-601]	ratingDashboard	Display a summary of all ratings, including foreman names and scores	[FPS-PACK-600]
[FPS-PACK-602]	foremanList	Display a list of foremen that users can select to rate or edit.	[FPS-PACK-600]
[FPS-PACK-603]	addRating	Display a form where users can give a new rating and comment for a selected foreman.	[FPS-PACK-600]

[FPS-PACK-604]	editRating	Allows users to update an existing rating and comment they previously submitted.	[FPS-PACK-600]
[FPS-PACK-701]	RegisterController	Manages all registration actions including handling user input, validating data and sending registration details to the database	[FPS-PACK-700]
[FPS-PACK-702]	ProfileController	Handles profile actions like viewing, updating and saving user profile information	[FPS-PACK-700]
[FPS-PACK-703]	ScheduleController	The controller that handles all working schedule management.	[FPS-PACK-700]
[FPS-PACK-704]	InventoryController	The controller is responsible for handling all inventory-related actions.	[FPS-PACK-700]
[FPS-PACK-705]	PaymentController	The PaymentController class handles all payment operations in the system. It manages the process of initiating, validating, and recording payments.	[FPS-PACK-700]
[FPS-PACK-706]	RatingController	Manages all rating actions such as adding, editing, viewing, and deleting ratings.	[FPS-PACK-700]
[FPS-PACK-801]	UserModel	Defines the data model for a user information	[FPS-PACK-800]

[FPS-PACK-802]	ScheduleModel	Stores data involving all the working schedule management data	[FPS-PACK-800]
[FPS-PACK-803]	InventoryModel	Stores data related to inventory part requests between workshops.	[FPS-PACK-800]
[FPS-PACK-804]	PaymentModel	Manages how payments work. It handles payment requests, checking payment details are correct, connecting payment systems, and keeping payment up to date.	[FPS-PACK-800]
[FPS-PACK-805]	RatingModel	Defines the data model for a rating	[FPS-PACK-800]
[FPS-PACK-901]	Firebase	A cloud tool that helps you build and run web or mobile apps with features like hosting, database, and more.	[FPS-PACK-900]
[FPS-PACK-902]	paymentGateway	A service that connects the user's bank to the merchant to make sure payments go through safely online.	[FPS-PACK-900]